

Introduzione

Introduzione

Il progetto si focalizza sul **motion capture markerless**, una tecnica di analisi del movimento umano che, a differenza dei sistemi tradizionali basati su **marker fisici applicati al corpo del soggetto**, opera esclusivamente a partire da sequenze video, eliminando la necessità di contatto diretto con il soggetto.

Obiettivi e Modelli Analizzati

L'obiettivo centrale della ricerca è stato l'analisi di diversi modelli di motion capture tra cui:

- **MoveNet (TensorFlow Hub)**: analizzato nelle varianti **Lightning**, **Thunder** e **Multipose**, basate su una pipeline di rilevamento dei keypoint ottimizzata per l'esecuzione in tempo reale.
- **MediaPipe (Google)**: utilizzato nella configurazione **Holistic** per una ricostruzione integrale (corpo, mani, volto) e in un approccio ibrido che integra **YOLOv8** per potenziare le capacità di *object detection* e tracking multi-soggetto.

Hardware e Vincoli di Esecuzione

L'analisi è stata condotta su architetture **Apple Silicon**, mettendo a confronto l'efficienza dei SoC (System on a Chip) **M1** e **M3**.

Una particolarità del progetto risiede nel vincolo software, a causa di limitazioni di incompatibilità tra le librerie di accelerazione (Metal) e **TensorFlow 2.21.0**, l'intera pipeline è eseguita esclusivamente sulla **CPU**.

Questa condizione trasforma il test in un'analisi comparativa dell'efficienza dell'**Instruction Set Architecture (ISA) ARM** e della gestione del multi-threading. In questo scenario, l'architettura **Unified Memory (UMA)** di Apple risulta determinante nel minimizzare i tempi di copia dei dati tra le fasi di pre e post-processing gestite dal processore.

Apple Silicon

Apple Silicon è una serie di **system-on-a-chip (SoC)** ovvero un circuito integrato (dove i vari transistori sono formati tutti nello stesso istante, a partire da un die di wafer in semiconduttore, che rappresenta il **core o nucleo** del processore) che in un **solo chip contiene un intero sistema che integra il processore centrale con un chipset e altri controller come la memoria RAM, circuiteria input/output**.

I SoC Apple contengono al loro interno una serie di componenti come:

- la **CPU**: il processore primario che si occupa dell'elaborazione parallela delle istruzioni aritmetiche e di calcolo;
- **GPU**: unità di elaborazione dedicata all'elaborazione parallela delle istruzioni grafiche tridimensionale;
- **NPU: unità di elaborazione dedicata al Neural Engine**, per l'apprendimento integrato (machine learning) e per la realtà aumentata;
- **ISP**: Image Signal Processor, unità di elaborazione dedicata all'elaborazione delle immagini raccolte dal sensore fotografico;
- **Mx**: unità di elaborazione dedicata al movimento (Motion Coprocessors), che si occupa della raccolta dati dei sensori integrati come accelerometro, giroscopio, bussola, barometro;
- **SEP**: (Secure Enclave Processor), unità di elaborazione dedicata alla protezione dei dati tramite chiavi crittografate.

SERIE M

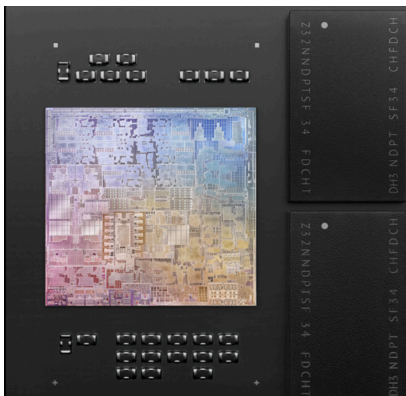
A differenza delle architetture tradizionali Intel, dove CPU e GPU hanno memorie separate, i chip della serie M adottano una **memoria unificata (Unified Memory Architecture, UMA)**, che al posto di avere una RAM per la CPU e una memoria video per la GPU, **usa un unico pool di memoria RAM ad alta velocità accessibile da tutti i componenti**, eliminando la necessità di copiare i dati tra le unità e riducendo significativamente la latenza tra gli stadi della pipeline di elaborazione.

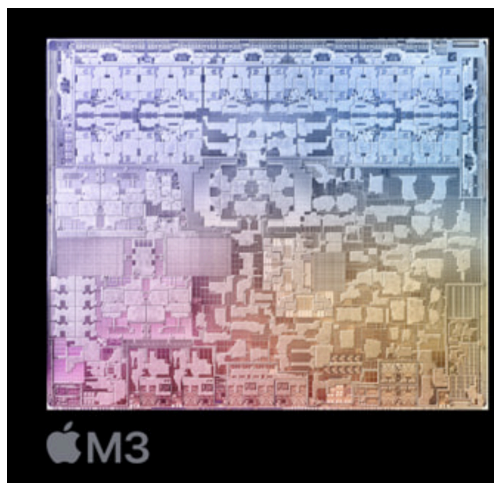
CHIP M1

Il chip è realizzato con tecnologia a **5 nanometri** e integra 16 miliardi di transistor. L'M1 usa **CPU a 8 core** (sono unità di elaborazione indipendenti all'interno della CPU capaci di eseguire istruzioni e calcoli) con **4 performance core (per i task pesanti come il post-processing dei keypoint)** e **4 efficiency core (che gestiscono i processi in background)**.

La **GPU** ha fino a 8 core (7 core per il macbook air M1 usato per l'analisi del progetto) ed è in grado di gestire quasi 25.000 thread simultaneamente con un throughput di 2,6 teraflop.

Il Neural Engine è in grado di eseguire 11.000 miliardi di operazioni al secondo.





CHIP M3

Il chip M3 è realizzato con tecnologia a 3 nanometri e integra 25 miliardi di transistor. La GPU di nuova generazione introduce la tecnologia Dynamic Caching, che consente un rendering fino a 2,5 volte più veloce rispetto all'M1. I performance core della CPU sono fino al 30% più veloci rispetto a quelli dell'M1, mentre gli efficiency core sono fino al 50% più scattanti, consentendo alla CPU nel complesso di raggiungere prestazioni di picco fino al 35% superiori usando la metà dell'energia. Il Neural Engine è il 60% più veloce rispetto alla famiglia M1, accelerando ulteriormente i flussi di

lavoro di Machine Learning mantenendo i dati elaborati sul dispositivo.

Architettura ARM e filosofia RISC

La serie M è una famiglia di sistemi su chip progettata da Apple sviluppata principalmente per computer Mac e successivamente per iPad segnando la transizione di Apple verso architetture basate su **ARM**. L'acronimo **ARM sta per Advanced RISC Machine**, perciò ingloba a sé il funzionamento **RISC (Reduced Instruction Set Computer)**, ovvero un'architettura a set di istruzioni ridotto con esecuzione diretta. Si tratta di processori generalmente a 64 bit con layout relativamente semplice e con design processuale efficiente.

Esecuzione e Registri

Un aspetto distintivo dell'architettura RISC, quindi ARM, è che ogni istruzione viene eseguita **interamente in un singolo ciclo di clock**, a differenza delle architetture CISC tradizionali dove, **ogni istruzione è divisa in microistruzioni che richiedono uno o più cicli ciascuna**.

Per supportare questo modello di esecuzione, **ARM dispone di un register file ampio**, tipicamente 16 registri visibili a 32 bit (legacy), che consente di mantenere i dati di lavoro direttamente nel processore, riducendo gli **accessi alla memoria**.

Mentre l'architettura ARM a 32-bit disponeva di 16 registri visibili, l'attuale architettura **ARM64** utilizzata da Apple offre:

- **31 Registri General Purpose (X0-X30)**: a 64 bit, utilizzati per operazioni matematiche e puntatori.
- **Stack Pointer (SP)**: per la gestione dello stack.
- **Link Register (LR/X30)**: per gestire i ritorni dalle funzioni.
- **Program Counter (PC)**: che punta alla prossima istruzione.
- **CPSR (Current Program Status Register)**: che monitora i flag di stato della ALU.

Questa struttura, unita a un design **superscalare**, permette ai chip Apple di mantenere un'efficienza energetica straordinaria pur garantendo prestazioni di calcolo elevate.

Componenti Hardware e Funzionalità avanzate

A livello architetturale, il processore ARM si distingue per una struttura snella ma ottimizzata, finalizzata alla **massimizzazione del throughput per ciclo di clock**. I componenti hardware fondamentali includono:

- **MAR (Memory Address Register) e MBR (Memory Buffer Register)**, i quali sono registri dedicati alla gestione dell'interfaccia con la memoria, rispettivamente per l'indirizzamento e il transito temporaneo dei dati.
- **Register File (Banco di Registri)**, ovvero un set di registri ad alta velocità che riduce la necessità di accedere alla memoria esterna, accelerando l'elaborazione dei dati locali.
- **Booth's Multiplier**, un'unità hardware specializzata che implementa l'algoritmo di Booth per eseguire moltiplicazioni complesse in modo rapido ed efficiente.
- **Barrel Shifter**, un componente critico posizionato in linea con la **ALU (Arithmetic Logic Unit)**.

Struttura fisica di un PC: CPU e FETCH - DECODE - EXECUTE

La base della struttura di un PC è data dalla **scheda madre**, dove si trova la **CPU (Central Processing Unit)**, il **chipset**, ovvero l'insieme di circuiti integrati per la comunicazione al suo interno, il **BUS**, i connettori vari per la memoria e I/O.

Un Bus è una struttura di interconnessione che collega più moduli, dove più dispositivi possono essere collegati ma solo uno per volta può utilizzarlo.

Nei calcolatori moderni distinguiamo:

- **BUS INTERNI**, all'interno di una singola unità funzionale.
NON standardizzati, ma dipendenti dalle unità che li sfruttano;
- **BUS ESTERNI**, estesi all'esterno dell'unità funzionale, **standardizzati**.

Un bus è costituito da un **data bus che trasporta dati ed istruzioni** e il quale può avere diverse ampiezze da 8 bit a 64 bit , il **address bus** che identifica le sorgenti o la destinazione dei dati e la sua **ampiezza che determina lo spazio di memoria indirizzabile** e il **control bus** il quale è volto alla trasmissione di informazioni di controllo e temporizzazione (tra cui il clock).

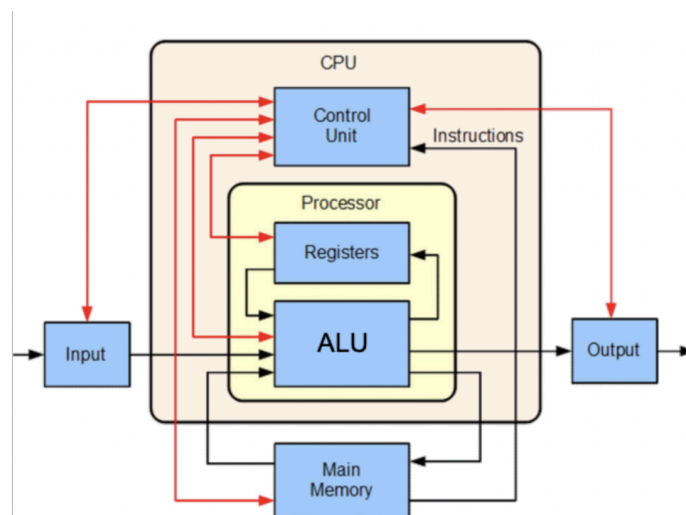
Le trasmissioni possono essere parallele, quindi costituite da un **filo per bit** o **seriali** costituite da un bit uno per volta.

Su Apple Silicon, i bus interni che collegano CPU, GPU e Neural Engine **non sono standardizzati e ottimizzati** per l'architettura specifica, garantendo trasferimenti più rapidi tra le unità di elaborazione rispetto a un sistema Intel tradizionale.

La **CPU** è implicata nell'esecuzione di operazioni logiche ed aritmetiche sui dati contenuti nei registri, si opera inoltre lo spostamenti di dati tra i registri e la memoria

Il ciclo **FETCH-DECODE-EXECUTE** è un ciclo elementare e naturale della CPU, dato dal **FETCH** che si occupa del caricamento dell'istruzione dalla memoria all'**IR (instruction register)**, viene incrementato successivamente il **PC (program counter)**, poi l'istruzione viene decodificata nella fase di **DECODE**. Se l'istruzione ha un dato della memoria, viene **calcolato l'indirizzo e caricato l'operando nel registro**. Nella fase di **EXECUTE** viene eseguita l'istruzione e lo **store memorizza il risultato**.

Ogni stadio ha il proprio compito specifico e opera in sequenza, ma grazie al parallelismo più frame possono trovarsi in **stadi diversi contemporaneamente**.



Pipeline di elaborazione

Parallelismo e Pipelining

Il concetto cardine dell'ottimizzazione dei processi è il **parallelismo**, il quale permette di superare i colli di bottiglia dell'esecuzione sequenziale.

Il parallelismo si manifesta a due livelli:

- a livello di **istruzione**: Ottenuto tramite il **pipelining**, dove **un'istruzione è suddivisa in stadi gestiti** in contemporanea su istruzioni diverse;
- a livello di **processore**: diversi processori lavorano congiuntamente sullo stesso problema. Esistono diverse architetture multiprocessore e multicore che lo sfruttano.

A temporizzare uno stadio di esecuzione è un **ciclo di clock**, il quale stabilisce ogni quanto un'istruzione possa completarsi.

Il tempo complessivo di esecuzione è detto **fattore di guadagno ed è pari al numero di stadi della pipeline**. Nella pratica tuttavia questo valore è sempre **inferiore, a causa di dipendenze tra istruzioni consecutive, stalli di pipeline e overhead di sincronizzazione tra stadi**.

La **latenza** è il tempo di **elaborazione dell'istruzione** e la **larghezza di banda del processore** è il **numero di istruzioni per secondo**.

Un ciclo di clock è dato dal suo periodo T e la frequenza ad esso correlata è uguale a $10^9/T$ e corrisponde alla larghezza di banda. Se il numero di stadi della pipeline è uguale ad n , allora la latenza è data da nT nanosecondi.

Architetture SUPERSCALARI

Le architetture superscalari sono composte da **più linee di pipeline in parallelo tra di loro**.

Le pipeline possono essere specializzate, contenenti alcune unità specifiche rispetto ad altre e in un'architettura di questo genere può presentarne **omogenee o disomogenee**:

- In un'architettura **omogenea**, il processore dispone di più **pipeline identiche** tra di loro e ogni pipeline è generalista e può eseguire qualsiasi tipo di istruzione.
- Nelle pipeline **disomogenee** o **specializzate**, il processore smista le istruzioni in base al tipo di pipeline.

L'aumento delle prestazioni tramite Pipelining e architetture superscalari garantisce un incremento di un fattore 5-10. Tuttavia per ottenere guadagni superiori è necessaria la presenza di più unità di elaborazione che operano secondo il modello **SIMD (Single Instruction Multiple Data)**, un approccio **Data Parallel** in cui tutti i processori eseguono la stessa sequenza di istruzioni su dati differenti e **MIMD (Multiple Instruction Multiple Data)**, un approccio Task Parallel che sfrutta architetture multiprocessore e multicore. Le unità di elaborazione lavorano in modo indipendente su compiti diversi. Poiché la pipeline di questo progetto è eseguita sulla CPU, essa ricade prevalentemente in questo modello di parallelismo.

Tensorflow: MoveNet

Tensorflow

Tensorflow è una piattaforma end-to-end, ovvero una piattaforma che copre l'intero ciclo di vita di un progetto di Machine Learning, dalla gestione dei dati grezzi fino alla messa in produzione di un modello finito.

Utilizza un approccio basato sull'apprendimento automatico il quale supporta il calcolo numerico basato su array multidimensionali (chiamati tensori), che fluiscono attraverso un grafo di operazioni matematiche. Questa struttura consente a TensorFlow di scalare in modo efficiente da un singolo processore a complessi sistemi distribuiti composti da migliaia di GPU o TPU.

Tensori

Il nucleo di TensorFlow è storicamente basato sul concetto di **grafo computazionale**, una **struttura dati diretta** in cui i nodi rappresentano **unità di calcolo** (oggetti **tf.Operation**) e **gli archi** rappresentano i dati (oggetti **tf.Tensor**) che fluiscono tra di essi.

La natura intrinseca di un tensore è la sua immutabilità, difatti una volta inizializzata, la **struttura non può essere modificata**, garantendo la **coerenza del grafo di calcolo** e **facilitando l'ottimizzazione dei flussi di dati attraverso i nodi operativi**.

La **classificazione dei tensori** avviene primariamente attraverso il concetto di **rango**, che identifica **il numero di dimensioni o assi posseduti dalla struttura**. TensorFlow opera su **array multidimensionali o tensori** rappresentati come oggetti **tf.Tensor**.

Gli attributi più importanti di un **tf.Tensor** sono la sua **shape** e il suo **dtype**. Con **Tensor.shape** viene indicata la dimensione del tensore lungo ciascuno dei suoi assi e con **Tensor.dtype** viene detto il tipo di tutti gli elementi del tensore. TensorFlow implementa **operazioni matematiche standard** sui tensori, oltre a molte operazioni specializzate per l'apprendimento automatico. L'esecuzione di calcoli di **grandi dimensioni sulla CPU**, può essere lenta, ma se configurato **correttamente Tensor Flow può utilizzare l'hardware dell'acceleratore come le GPU per eseguire operazioni molto rapidamente**.

Variabili dei tensori

Una **rete neurale** è composta da **strati di neuroni**, una **successione di trasformazioni matematiche**, il cui compito è prendere un dato in ingresso e trasformarlo in un risultato, ogni connessione tra i neuroni ha un peso (W). I pesi in TensorFlow non sono semplici numeri volatili, ma sono saltati dentro oggetti speciali chiamati **tf.Variable**.

Se i tensori forniscono la struttura per il transito dei dati, le variabili (`tf.Variable`) ne rappresentano lo stato mutabile.

Le **variabili** sono inoltre classificate in base alla loro "**addestrabilità**" (`trainable`), ovvero in base ad un parametro i cui valori vengono aggiornati durante la fase di addestramento. Questi parametri sono tipicamente i "**weights**" (o **pesi**) che determinano l'importanza di un input rispetto a un neurone e i "**biases**" i quali permettono di traslare la funzione di attivazione per adattarsi meglio ai dati. Questo è fulcro della **back-propagation**, la quale calcola il **gradiente di perdita (Loss)** rispetto ad ogni parametro. Per impostazione predefinita, ogni `tf.Variable` è creata con il parametro **`trainable=True`**, indicando esplicitamente al sistema che essa deve essere inclusa nel calcolo dei gradienti. Il sistema calcola quindi la derivata parziale:

$$\frac{\partial Loss}{\partial w}$$

Differenziazione automatica.

Tensorflow implementa la **differenziazione automatica**, ovvero la capacità di un modello di apprendere dai propri errori dipende dalla precisione con cui può calcolare l'impatto di ogni **singolo peso sull'errore complessivo (Loss)**. TensorFlow realizza questo compito attraverso il meccanismo della differenziazione automatica, implementato principalmente tramite **`tf.GradientTape`**.

Durante il **forward pass** (passaggio in avanti), il "**nastro**" (tape) agisce come un registratore dinamico, annotando ogni operazione eseguita e i relativi tensori di input.

Il **calcolo dei gradienti** avviene tramite la **modalità inversa**, computazionalmente più efficiente per le funzioni che **mappano molti input** (i **parametri** del modello) **verso un singolo output scalare** (la **loss**):

- Una volta completato il forward pass, chiamando il **metodo `gradient`**(`target`, `sources`), il sistema **percorre il nastro all'indietro**, applicando la **REGOLA DELLA CATENA** (chain rule) per **derivare** i contributi di errore rispetto ai parametri sorgente.

La **regola della catena** è l'algoritmo matematico che permette di scomporre la derivata di una funzione composta in un **prodotto di derivate più semplici**. In una rete neurale, l'errore (**Loss**) è il risultato finale di una lunga sequenza di operazioni stratificate.

$$\frac{\partial Loss}{\partial w} = \frac{\partial Loss}{\partial output} \cdot \frac{\partial output}{\partial attivazione} \cdot \frac{\partial attivazione}{\partial w}$$

Il **peso** (w) moltiplica l'input per generare una somma ponderata (spesso chiamata pre-attivazione). La **funzione di attivazione** elabora questo valore per produrre l'**output** del neurone. L'output diventa l'input per il livello successivo, propagandosi fino al calcolo della Loss.

Tensorflow: MoveNet

MoveNet è un modello ultra veloce e accurato **che rileva 17 punti chiave di un corpo**, attraverso il metodo "**bottom-up**" sviluppato da Google. A differenza dei modelli "top-down", che prima identificano le persone nel frame tramite un detector di oggetti e poi stimano le giunture all'interno dei riquadri rilevati, **MoveNet localizza i keypoints direttamente**, riducendo drasticamente il carico computazionale e la latenza.

Il modello è offerto da **TF Hub** con due varianti: **Lightning** e **Thunder**. Il modello Lightning è sviluppato per applicazioni critiche per la latenza, mentre Thunder è destinato ad applicazioni che richiedono un'elevata precisione.

Entrambi i modelli funzionano più velocemente del tempo reale (30+ FPS) sulla maggior parte dei dispositivi.

Architettura del Modello e Influenza di CenterNet

L'architettura di MoveNet è composta da due componenti principali: un **estrattore di caratteristiche (BACKBONE)** e un **insieme di teste di predizione specializzate**.

L'impatto di MoveNet sul campo del "human pose estimation" :

- **Leggerezza dell'architettura:** MoveNet utilizza un'architettura ottimizzata per la velocità e costo computazionale ridotto.
- **Alta accuratezza:** nonostante la sua natura leggera, MoveNet è capace di detectare le giunture del corpo.
- **Integrazione di TensorFlow Lite:** dato che è costruito per TensorFlow Lite permette l'utilizzo di una vasta varietà di strutture.
- **Multi-Person Pose Tracking:** MoveNet è capace di detectare e stimare le pose di coppie di individui, rendendolo utilizzabile per situazioni in cui vengono inclusi più esseri umani.
- **Robustezza:** la versione è progettata per resistere abbastanza bene alle variazioni degli apparecchi di illuminazione, dell'abbigliamento e degli angoli di visione, anche se le condizioni estreme possono comunque rappresentare delle sfide.

MoveNet adotta un approccio backside-up per la stima della posa di più uomini o donne. La sua struttura comprende due componenti principali:

- Estrattore di feature (**backbone**): MoveNet utilizza il **MobileNetV2**, una **rete neurale convoluzionale leggera** che utilizza convoluzioni separabili in profondità per minimizzare il numero di parametri e le operazioni in virgola mobile.
- **Predizione delle teste**: insieme all'estrazione vi sono numerose predizioni delle teste:
 - **Mappa di calore del centro della persona (heatmap)**: prevede il centro geometrico di tutte le persone;
 - **Disciplina di regressione dei keypoints**: prevede l'intero **set di keypoints per ogni corpo**, utilizzato per raggruppare i keypoints in pose individuali.
 - Mappa termica dei keypoints delle persone (heatmap): prevede la **posizione di tutti i keypoints**, indipendentemente dai singoli momenti.
 - 2D in base al campo di offset del keypoint: prevede gli **offset dei vicini** per perfezionare la regione sub-pixel di ciascun punto chiave.

Lo schema di previsione di MoveNet è stimolato attraverso **CenterNet**, un modello di **rilevamento di oggetti** che prevede il punto medio degli oggetti. Tuttavia, MoveNet include modifiche specifiche per ottimizzare la stima della posa, migliorando ogni velocità e precisione.

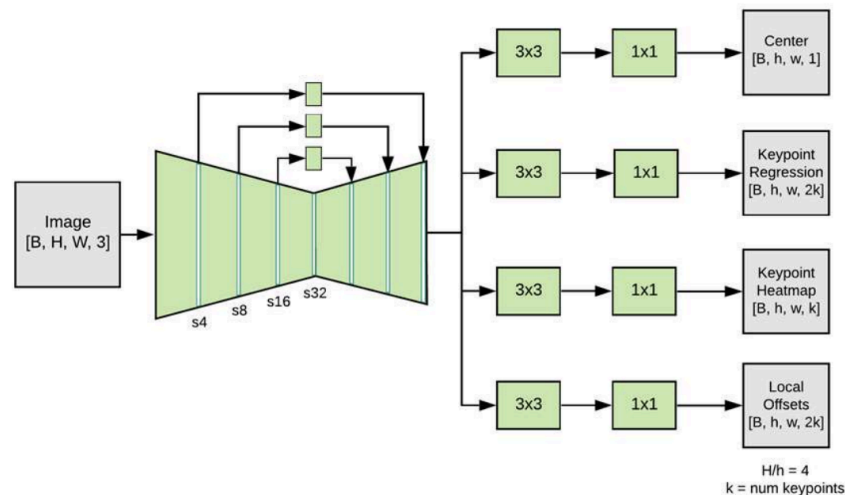
Altri principali metodi di approccio **bottom-up** che utilizzano previsioni basate su **heatmap** per la stima della posa umana sono:

- **Convolutional Pose Machines (CPM)**: questi modelli utilizzano una sequenza di reti convoluzionali per **raffinare progressivamente le previsioni sulle posizioni delle articolazioni del corpo**, migliorando la precisione a ogni stadio della rete.
- **Part Affinity Fields (PAF) in OpenPose**: i **PAF sono campi vettoriali** utilizzati per **associare tra loro i componenti** (keypoints) rilevati nel frame, permettendo di raggrupparli correttamente come appartenenti alla stessa persona. Un approccio simile è adottato dal campo di regressione dei punti chiave di **MoveNet**, che svolge una funzione analoga nel processo di raggruppamento dei keypoint.
- **HigherHRNet**: questa architettura estende il modello HRNet adattandolo a una **tecnica bottom-up**. Attraverso l'apprendimento di rappresentazioni scale-aware (sensibili alla scala) per il rilevamento e il raggruppamento dei keypoints, HigherHRNet ottiene risultati allo stato dell'arte, specialmente in scenari complessi e scene affollate.

Varianti MoveNet

MoveNet non è un singolo modello, ma una famiglia di architetture ottimizzate per diversi profili di prestazioni e requisiti applicativi. La scelta tra le varianti Lightning, Thunder e MultiPose determina la latenza complessiva del sistema e la sua capacità di operare in ambienti complessi.

Implementazione



Nella scatola grigia viene rappresentata l'immagine di input grezza con le sue dimensioni (batch size B, altezza H, larghezza W, 3 canali RGB).

L'input si alimenta in una struttura centrale a forma di **imbuto**. Questa struttura rappresenta la **rete neurale a collo di bottiglia** che estrae le feature. Le scatole verdi rappresentano i **blocchi di elaborazione**. Le etichette s4, s8, s16 e s32 indicano i **livelli di ridimensionamento o risoluzione delle feature**.

Si notano le **frecce di bypass (skip connections)** che collegano **livelli a risoluzione più alta a livelli a risoluzione più bassa** per **preservare** le informazioni spaziali.

Dopo l'ultimo livello della **backbone**, vi sono quattro rami di output paralleli, in cui viene effettuata un'operazione di convoluzione 3x3 seguita da una 1x1.

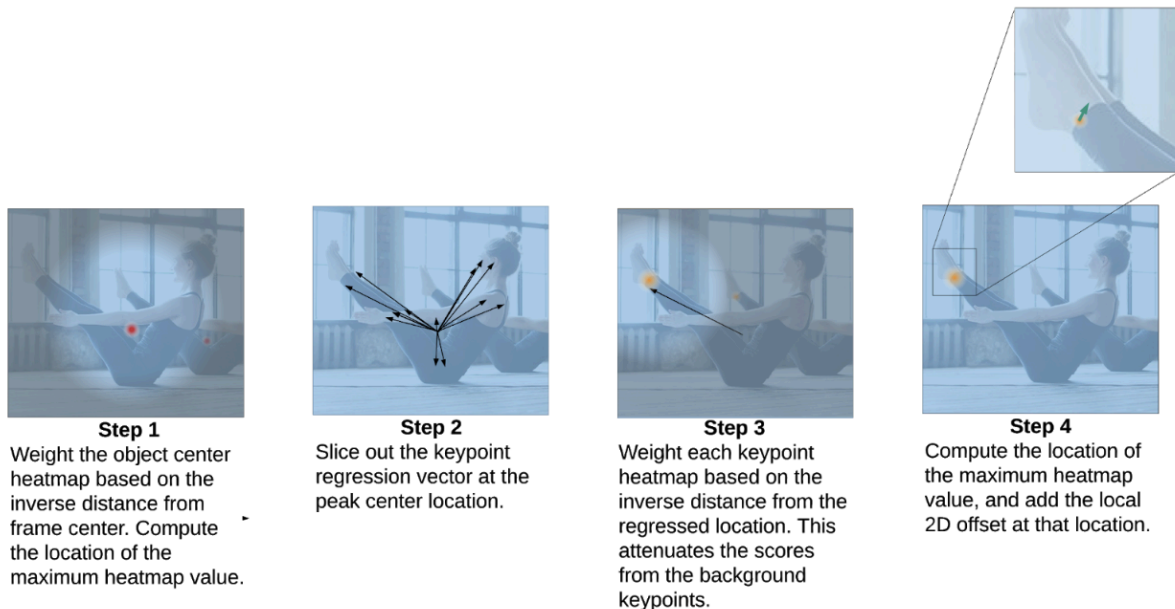
Questi rami portano ai 4 output finali:

- **Center [B, h, w, 1]** : il modello genera una mappa (heatmap) per individuare il centro geometrico della persona.
- **Keypoint Regression [B,h,w,2k]** : una volta trovato il centro, il modello partendo dal pixel del centro, mappa la regressione dei keypoints che fornisce delle previsioni iniziali (annotato con 2k canali, dove 2k = coordinate x, y per k keypoints);
- **Keypoint Heatmap [B, h, w, k]** : la heatmap per individuare le posizioni

precise dei keypoints

- **Local Offsets [B, h, w, 2k]** : la mappa degli offset locali per raffinare ulteriormente le coordinate dei keypoints.

Tutte queste predizioni sono eseguite in parallelo, vengono eseguiti le seguenti sequenze di operazioni:



1. La **persona al centro della heatmap** è utilizzata per identificare il centro di tutti gli individui presenti nel frame, definiti dalla media aritmetica di tutti i punti associati ad una persona.
2. Un set iniziale di keypoints per la persona viene prodotto suddividendo **l'output della regressione dei keypoints** dal pixel corrispondente al centro dell'oggetto. Poiché si tratta di una previsione centrata che deve operare su scale diverse la qualità dei keypoints regrediti non sarà molto accurata.
3. Ogni pixel nel heatmap degli keypoint è moltiplicato dal peso il quale è inversamente proporzionale alla distanza che corrisponde al keypoint regredito. Questo assicura che non vengano accettati i keypoints da parte delle persone in background.
4. Il **set finale di previsioni dei keypoints** viene selezionato recuperando le coordinate dei **valori massimi della mappa di calore** in ciascun canale dei keypoints. Le previsioni di offset 2D locali vengono quindi aggiunte a queste coordinate per fornire stime raffinate.

MoveNet Lightning e Thunder

Il modello MoveNet ha due varianti principali:

- **MoveNet.SinglePose.Lightning**: un modello con bassa capacità che può performare con medie maggiori a 50 FPS nella maggior parte di laptop ottenendo una buona performance.

- **MoveNet.SinglePose.Thunder**: un modello ad alte capacità che può ottenere migliori predizioni, performando comunque con medie maggiori a 30 FPS.

Input e Output

Lightning opera su un'immagine di input quadrata di 192 x 192 pixel. Questa risoluzione contenuta riduce la **mole di dati da processare** in ogni strato della rete neurale. La capacità del modello è calibrata utilizzando un **MOLTIPLICATORE DI PROFONDITÀ** pari a 1.0, il che significa che il numero di filtri convoluzionali segue la specifica standard di MobileNetV2. Questa configurazione permette al modello di superare i 50 FPS sulla maggior parte di computer portatili. L'output è rappresentato da un float32 con un tensore di dimensione [1, 1, 17, 3].

Thunder rappresenta la **variante ad alta capacità** destinata a scenari in cui la fedeltà delle coordinate articolari è prioritaria rispetto al minimo risparmio di millisecondi. Sebbene rimanga un modello capace di prestazioni in tempo reale (>30 FPS), la sua struttura interna è sensibilmente più densa e complessa.

La differenza più evidente risiede nella risoluzione dell'input, che viene incrementata a **256x256** pixel.

Questo aumento della densità dei dati in ingresso permette al modello di catturare dettagli più fini, essenziali per distinguere articolazioni parzialmente sovrapposte o per monitorare utenti situati a una distanza maggiore dalla telecamera.

A supporto di questa maggiore risoluzione, Thunder utilizza un **moltiplicatore di profondità di 1.75**, aumentando il numero di parametri e la larghezza della rete.

Questo incremento di capacità consente a Thunder di apprendere rappresentazioni più ricche e resistenti a condizioni di illuminazione difficili o pose estreme.

Metodologia di Addestramento e Dataset "Active"

L'efficacia di MoveNet in situazioni di movimento intenso deriva non solo dalla sua architettura, ma anche da una strategia di addestramento.

Mentre il **dataset COCO** fornisce una base solida per le **pose quotidiane**, il **dataset "Active" sviluppato da Google**, offre una collezione interna di immagini annotate provenienti da video di YouTube focalizzati su yoga, allenamenti HIIT e danza.

Algoritmi di Cropping

Il funzionamento del cropping segue parametri rigorosi, infatti il sistema verifica se **almeno una spalla e l'anca sono rilevate con un punteggio di confidenza**

superiore a 0.2. Se il torso è visibile, il centro del ritaglio viene fissato al punto medio tra le due articolazioni delle anche. La dimensione del ritaglio è calcolata come il massimo tra 1.9 volte la distanza del torso e 1.2 volte la distanza massima tra tutti i 17 punti chiave rilevati. Se il modello perde traccia del soggetto, a causa del punteggio di confidenza troppo basso, l'algoritmo ripristina la visualizzazione a pieno campo per tentare un nuovo rilevamento.

MoveNet MultiPose

Con caratteristiche del tutto analoghe dalla sua controparte SinglePose, MoveNet **MultiPose** si differenzia per la possibilità di detectare **più soggetti contemporaneamente** mantenendo un'esecuzione in tempo reale.

Multipose viene implementato utilizzando come base MoveNet in variante Lightning, affinché le performance durante la rilevazione di più soggetti non possano rischiare cali prestazionali più consistenti.

L'output rilasciato con MultiPose è un tensore di forma **[1, 6, 56]**, con:

- **1** : La batch size;
- **6** : Il **numero massimo** di soggetti che il modello riesca a rilevare contemporaneamente;
- **56 (51+5)** : Sono 51 i **keypoints di TRE BOUNDING BOXES** principali da predictare.

I rimanenti 5 elementi indicano la **regione della bounding box** (in coordinate normalizzate) : **ymin** , **xmin** , **ymax** , **xmax** , **score** (score indica il **valore di confidence**).

L'ordine dei 17 keypoints, condiviso per tutti i modelli MoveNet è : *[naso, occhio-sinistro, occhio-destro, orecchio-sinistro, orecchio-destro, spalla-sinistra, spalla-destra, gomito-sinistro, gomito-destro, polso-sinistro, polso-destro, fianco-sinistro, fianco-destro, ginocchio-sinistro, ginocchio-destro, caviglia-sinistra, caviglia-destra]*.

Gli indici dei keypoint vanno da 0 a 16.

MediaPipe

MediaPipe

MediaPipe è un framework (un blocco di codice pre-scritto che fornisce un'architettura per la costruzione di un programma) open-source sviluppato da Google che permette l'inserimento di modelli Machine Learning volti all'**analisi di video, immagini e audio in tempo reale**.

Si chiamano Solutions, la serie di librerie e di strumenti integrabili del progetto Mediapipe.

Queste librerie forniscono la funzionalità di base per ogni "soluzione":

- **MediaPipe Tasks**: comprendono le **API** (Application Programming Interfaces, dei set di regole e protocolli che permettono a differenti applicazioni software di comunicare, scambiare dati e condividere funzionalità) e librerie multiplatforma.

Mediapipe Tasks fornisce dunque l'interfaccia di programmazione di Mediapipe che include una serie di librerie per l'utilizzo di soluzioni di **ML** (Machine Learning) con una quantità ridotta di codice

- **Modelli Mediapipe**: si tratta dei modelli preaddestrati, pronti all'uso, i quali, tramite il **Mediapipe Model Maker** sono possibili personalizzabili e riaddestrabili.

Il Model Maker **rimuove gli ultimi livelli di un modello esistente per ricrearli con dei nuovi dati**. L'addestramento di un modello con un Model Maker in genere **riduce le dimensioni** di un modello e li modifica affinché siano più ADATTI per l'applicativo desiderato.

Quando un modello viene addestrato con dei nuovi dati, è necessario che sia addestrato con almeno 100 campioni della classe da analizzare. Nel nostro caso non è stato svolto un riaddestramento, ma sono state sfruttate le risorse principali che il framework base aveva da offrire.

Del framework, è stata implementata la **pipeline HOLISTIC**, da "**olistico**" (approccio globale che considera un sistema (persona, organismo, problema) nella sua interezza, piuttosto che come somma di singole parti), che gestisce in background dei diversi modelli di ML distinti e separati:

- **Pose Landmark Model**: il modello che **traccia** uno scheletro e le **articolazioni principali** del soggetto analizzato;
- **Face Landmark Model**: il modello che crea una **maschera 3D** del viso del soggetto;
- **Hand Landmark Model**: si tratta dei modelli volti alla **mappatura** esclusiva delle **dita delle mani**.

Con la Pipeline Holistic viene usato il modello **Pose** sull'intera immagine, analizzando spalle e braccia dove si trovano presumibilmente la testa e i polsi.

L'immagine analizzata viene **ritagliata** in piccoli pezzi e alcuni di questi, sono passati ai modelli **Face** e **Hand**.

Mediapipe Holistic

La pipeline Holistic consiste in una pipeline con componenti di posa, faccia e mani **ottimizzate** affinché vi sia un **minimo trasferimento di memoria** tra le inferenze **BACKEND** (Elaborazione direttamente **sul dispositivo** dell'utente. In questo caso si intende **backend** per il motore hardware locale che svolge i calcoli senza dover inviare nulla ad un server).

Holistic prevede che vi siano **543 keypoints** per l'analisi di singoli soggetti umani a corpo intero con:

- **33** keypoints per la **posa**;
- **21** per **ciascuna mano** (42);
- **468** esclusivamente per il viso.

La Holistic pipeline integra dunque diversi modelli per pose, mani e faccia, ciascuno singolarmente ottimizzato per il proprio scopo.

Nel caso del **Pose Estimation Model**, questo richiede un input con un frame a risoluzione fissa **non necessariamente elevato** (256x256). Ovviamente maggiore sarà la qualità di risoluzione dell'input ricevuto e più alta sarà l'**accuratezza** nella **stima** svolta dal modello.

BlazePose

Mediapipe Holistic stima le pose umane sul **BlazePose's pose detector**, una rete neurale con primo rilascio nella prima metà del 2020, che produce 33 keypoints per soggetto prevedendo il possibile posizionamento (le **coordinate**) delle articolazioni e dei punti di interesse del soggetto passo passo.

Limite nonché requisito principale di BlazePose, è la **necessità** che il volto della persona analizzata sia **SEMPRE** perfettamente visibile. Su base infatti della presenza del viso del soggetto, viene svolta una previsione dei parametri di allineamento aggiuntivi specifici della persona come **il punto medio tra i fianchi della persona e la dimensione del cerchio che circonda la persona intera e inclinazione**.

Rispetto alle topologie OpenPose o banalmente Kinect, ciò che viene proposto con il **Pose Estimation Model** basato su BlazePose, è l'uso del **numero minimo essenziale** di keypoints, i quali vengono accuratamente allineati al soggetto.

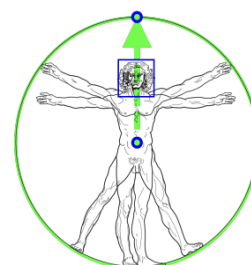
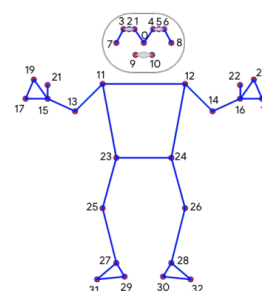


Figure 2. Vitruvian man aligned via our detector vs. face detection bounding box. See text for details.



Il training dataset utilizzato per lo sviluppo della rete neurale consistette in più di 60K immagini con uno o più soggetti in pose comuni e 25K immagini di **singoli soggetti** coinvolti in scene legate a contesti di attività fisica, aerobica e non.

Fondamenta di MediaPipe: l'impiego di MobileNetV2 nella computer vision

Sebbene Mediapipe utilizzi dei modelli ottimizzati della classe “Blaze”, come BlazePose, la base di questi è l'estensione di alcuni elementi introdotti da MobileNetV2.

MediaPipe infatti sfrutta i concetti di “**Inverted Residuals**” e “**Linear Bottlenecks**”, riadattati affinché vi fosse una **riduzione del costo computazionale** e dell'**impatto sulla memoria** che con 500+ landmarks in tempo reale sarebbero stati eccessivi.

Le **DEPTHWISE SEPARABLE CONVOLUTIONS** (le convoluzioni separabili in profondità) sono degli elementi chiave per numerose architetture efficienti di reti neurali. L'idea base dietro questo concetto è quello di rimpiazzare un operatore di convoluzione piena con una sua **versione FATTORIZZATA** che **divide la convoluzione di dati** in due layer separati:

- Il primo layer è una **depthwise convolution** (ossia una **convoluzione in profondità**) e svolge un filtraggio leggero applicando un singolo filtro convoluzionale per canale di ingresso;
- Il secondo layer è una **convoluzione 1x1**, detta infatti **pointwise**, che è responsabile della costruzione di nuove feature attraverso il calcolo di combinazioni lineari dei canali di input.

Generalmente una convoluzione normale prende un **tensore** (un blocco di dati) di una certa altezza **h** e una certa larghezza **w** con un numero di canali di input **d** e **ne applica un kernel** di dimensioni **kxk** per produrre un blocco di dati in uscita. Il problema di una convoluzione regolare si pone con immagini ad alta risoluzione, dove il numero di operazioni necessarie crescerebbe drasticamente.

Dividendo l'operazione in passaggi separati il costo computazionale si riduce 8/9 volte rispetto alle condizioni standard.

Il **BOTTLENECK LINEARE** affronta un problema critico, ossia quello della compressione dei dati. Infatti con il concetto di Linear Bottleneck si cerca di ovviare alla possibilità che con la compressione le informazioni vengano distrutte:

- Consideriamo la **funzione ReLU** (ossia **Rectified Linear Unit**) come una delle funzioni di attivazione più utilizzate nel campo del Machine Learning, la quale restituisce un input se positivo e 0 se è ≤ 0 .

Si tratta di una funzione inserita nei livelli nascosti delle reti neurali per **introdurre una NON LINEARITÀ**.

- Matematicamente la ReLU, se applicata in un livello della rete con pochi canali, rischia di distruggere irrimediabilmente un carico cospicuo di informazioni. Per evitare ciò, è necessario **RIMUOVERE la ReLU** nei punti detti “**colli di bottiglia**”, ossia i più “**STRETTI**”.
- All'interno dei bottlenecks, i dati subiscono solo delle trasformazioni lineari, mantenendo dunque il proprio potere rappresentativo senza perdere informazioni.

Il fulcro dell'efficienza computazionale dei modelli alla base di Mediapipe risiede negli **INVERTED RESIDUALS (Blocchi Residuali Invertiti)**.

Con un BLOCCO RESIDUALE l'input **NON è obbligato a passare** esclusivamente tramite delle convoluzioni, ma viene **sdoppiato**:

- Da una parte passa attraverso le convoluzioni;
- Dall'altra una **copia esatta** dell'input **SALTA COMPLETAMENTE LE CONVOLUZIONI** tramite uno shortcut detta “**skip connection**”.

Alla fine del blocco, i dati elaborati con la convoluzione e la copia vengono **SOMMATI INSIEME**, permettendo alla rete di imparare solo un **residuo** dato dalla effettiva convoluzione, tenendo a sé le informazioni originali copiate.

Contrariamente ai blocchi residuali standard che connettono strati ad alta dimensionalità (a **ELEVATA profondità di dati**, con tanti canali del blocco di dati), con gli **inverted**, inseriscono le connessioni di shortcut **DIRETTAMENTE TRA I COLLI DI BOTTIGLIA** a bassa dimensionalità.

Il livello di espansione intermedio rimane uno spazio **puramente transitorio**.

Pose Estimation con l'approccio CenterNet

Da tenere in considerazione è l'approccio metodologico di rilevamento impiegato tramite l'impiego di algoritmi di **Pose Estimation** come quelli presentati.

A differenza infatti delle metodologie di Object Detection come **YOLO** (il quale è stato comunque impiegato all'interno del progetto tramite una sua integrazione nel programma di Pose Estimation svolto con Mediapipe), fondate sull'identificazione tramite una **BOUNDING BOX**, ossia una vera e propria **delimitazione a scatola** dell'oggetto considerato, nel caso corrente definiamo il paradigma utilizzato fondato su una mappatura dei soli “**punti salienti**” dell'elemento rilevato.

Ogni oggetto infatti è modellato come un **singolo punto, quello centrale** dell'ipotetico bounding box di cui farebbe parte e poi successivamente mappato tramite una **HEATMAP**, ovvero una “**mappa di calore**”, dove i punti più **CALDI** corrispondono ai “**centri**” degli oggetti identificati.

Attraverso un approccio a punto centrale, il modello considera le 17 articolazioni umane stimando rispetto a quello stesso centro.

Anche nel caso in cui più elementi di natura differente, soggetti alla detection si **sovrapponessero** o **collidessero**, la predizione con CenterNet verrebbe meno in meno dello **0.1% delle casistiche**.

Analisi nello specifico del framework Mediapipe

A livello architetturale, MediaPipe non appare come un programma a flusso lineare, ma come un **GRAFO**, dove i nodi, detti **CALCULATORS** all'interno della stessa documentazione ufficiale, ricevono zero o più stream di **PACKETS** (blocchi **immutabili** di dati) e svolgono ciascuno un compito differente dall'altro.

I Packets essendo immutabili sono passati **per riferimento** a ciascun Calculator tramite dei puntatori.

Ciascun nodo del grafo è connesso ad un altro tramite uno **STREAM** che trasporta una sequenza di packets i cui **timestamp CRESCONO IN MANIERA MONOTONA**, ovvero per ciascun packet passato, il suo successivo dovrà necessariamente avere un timestamp **maggiore** (i dati non possono decrescere temporalmente).

Ciò si presenta come una chiave all'interno dei processi esecutivi del framework, poiché viene obbligato un passaggio ORDINATO dei dati e dunque una sincronizzazione appropriata delle informazioni in passaggio.

I Calculators sono altamente customizzabili, partendo dalla stessa base (la classe Calculator) con **4 metodi essenziali**:

- **GetContract()**: permette all'autore di specificare i **tipi** di input e output desiderati;
- **Open()**: viene chiamato dal framework una sola volta per ogni esecuzione del grafo e avviene prima che i flussi di dati scorrano attraverso i vari nodi. A tutti gli effetti, il metodo Open() **inizializza il Calculator** e assicura che i **SIDE PACKETS** (ossia dei blocchi di dati costanti definiti come parametri di configurazione, costanti matematiche etc..) siano già disponibili all'avvio. Ovviamente un errore durante l'Open() può terminare la run del grafo:
- **Process()**: metodo chiamato ripetutamente quando almeno uno stream di input ha un packet disponibile. Possono essere chiamati più Process() **in parallelo** se sono avviate più esecuzioni simultaneamente.
- **Close()**: metodo chiamato quando tutti gli stream di input sono chiusi o quando viene rilevato un errore durante i Process(). Nessuno stream o input è disponibile durante il Close(), ma rimangono a disposizione i Side Packets ed è possibile ancora stampare le uscite.

Grafi in MediaPipe

Il grafo è la struttura dove avviene l'intera elaborazione di dati. Si costituisce dei vari Calculator che collegati tra di loro consentono lo scorrimento dei Packets.

I dati possono entrare nel grafo:

- Tramite **nodi SORGENTE** che generano dei pacchetti spontaneamente;
- Tramite **GRAPH INPUT STREAMS** che "riempiono" i packets progressivamente (un esempio è l'acquisizione dei frame della webcam del dispositivo in diretta).

Saranno i **SINK NODES** (o nodi pozzo) a **raccogliere** i dati e trascriverli in varie destinazioni, come file, memory buffer etc.

Prima di partire con l'esecuzione, il framework verifica che ogni flusso di dati abbia una sola sorgente e che i dati siano compatibili con i contratti stabiliti.

Quando tutti i nodi hanno terminato con il metodo Close(), il grafo termina il proprio processo.

Definiamo inoltre come **GraphConfig** una specifica di MediaPipe che descrive la topologia e la funzionalità del grafo all'interno del framework. Nella specifica, un nodo del grafo rappresenta **un'istanza di un calcolatore in particolare**.

Modularizzare una GraphConfig in **sotto-moduli** è possibile come **Subgraph** (un sottografo) le cui specifiche sono modificate per adattarsi alle funzionalità necessarie. A tutti gli effetti, l'implementazione Holistic, implica a sé l'uso di numerose serie di sottografi (ad esempio per le mani, il viso, la posa stessa e le specifiche di ciascuno di essi).

Implementazione

Scheduling e Timestamp di sincronizzazione

MediaPipe sfrutta un sistema di **Scheduling** per pianificare l'ordine di lavoro dei vari Calculators. Ogni nodo infatti avrà uno **stato** tra **NOT READY**, **READY** e **RUNNING**. Quando i dati sono disponibili ed un nodo risulta READY, questo viene **inserito in coda di esecuzione**.

Importante da considerare è la reattività dei nodi:

- I **nodi SORGENTE** sono **sempre in ready** fino a che i dati da fornire non terminano;

- Gli altri nodi invece diventano ready quando presentano degli input da processare.

I nodi più **VICINI alla zona di OUTPUT del grafo** presentano una **MAGGIORE PRIORITÀ DI ESECUZIONE** rispetto agli altri.

L'esecuzione del grafo è **DECENTRALIZZATA**, perciò **non esiste un clock globale** a cui i nodi si affidano. Per questa ragione, è possibile che i dati vengano processati allo stesso tempo **pure se con timestamp differenti**.

I timestamp presentano infatti un ruolo cruciale nell'esecuzione poiché fungono da **SYNCHRONIZATION KEYS**: se un nodo deve unire due flussi, per la **Input Policy** (che definisce le regole e linea guida per la gestione degli input di dati) è necessario che i pacchetti con gli **STESSI TIMESTAMP siano elaborati SEMPRE INSIEME, indipendentemente dall'ordine di arrivo**.

Tramite questa sincronizzazione, avviene dunque una fase di riordinamento indiretto dei dati.

Ogni stream ha un **TIMESTAMP BOUND** (un limite di timestamp), ossia il **limite più basso** consentito per l'inserimento di un nuovo packet al suo interno.

La Input Policy di base (Default) fornisce una base **totalmente DETERMINISTICA** (in cui il risultato è predeterminato da cause specifiche) con le seguenti garanti:

- Se i packets con gli stessi timestamp sono forniti in stream multipli, **saranno sempre processati insieme**;
- Gli input set sono processati con i timestamp **crescenti MONOTONI**;
- **Nessun packet VIENE SCARTATO**;
- Il nodo diventa **"ready"** per processare i dati il prima possibile, appena le condizioni precedenti sono soddisfatte.

Per mantenere il rispetto di queste normative, è da definire il concetto di **SETTLED TIMESTAMP** (timestamp consolidato), con il quale si **riconosce irrevocabilmente uno stato** in un determinato istante. Il sistema infatti ha **piena certezza** se esiste un pacchetto o meno per un dato timestamp.

Supporto della GPU e di OpenGL

Mediapipe consente di far lavorare i nodi base sulla CPU **con quelli basati sulla scheda video**. I singoli nodi possono essere scritti con **API diverse** che consentono di sfruttare i possibili vantaggi della piattaforma su cui sono impiegati.

I nodi della GPU in MediaPipe usano un **tipo di buffer OPACO**, ovvero un'area di memoria il cui layout interno è **nascosto all'utente**, richiedendo l'accesso tramite le funzioni API **e non la manipolazione diretta dall'utente**.

Quando un nodo GPU necessita di accedere al buffer, richiede al sistema una **visione EFFIMERA**, la quale verrà meno alla fine del controllo.

Il framework inoltre supporta l'implementazione di **OpenGL (OPEN GRAPHICS LIBRARY)**, un API costruita per il rendering di vettori grafici 2D e 3D.

(Utile ai fini del progetto, MediaPipe supporta Metal in iOS).

Affinché possa avvenire una renderizzazione efficiente, con un frame rate più elevato e **senza rallentamenti forzati**, è possibile runnare OpenGL con **contesti MULTIPLI**. Dato che un singolo contesto corrisponde ad una coda sequenziale di comandi, l'uso dello stesso contesto per inferenze e rendering, **porterebbe ad un rallentamento forzato del frame rate**.

In aggiunta, è da considerare che **NON è possibile accedere a un contesto OpenGL da più thread contemporaneamente**.

Poiché il cambio di contesto provoca da sé dei rallentamenti, soprattutto nei dispositivi Android, MediaPipe assegna dei **thread indipendenti** a ciascun contesto, che invia comandi alla GPU **in modo ASINCRONO**.

Per garantire però una sincronia tra contesti, OpenGL sfrutta il concetto della **SYNC FENCE** (recinto di sincronizzazione):

- Durante l'esecuzione di un contesto (A), un contesto (B) nel suo flusso di comandi, presenterà un **Wait**, che garantisce alla CPU che i suoi comandi siano eseguiti **al termine di tutti i comandi di (A) interni alla recinzione** (la fence). Ciò permette al secondo contesto di svolgere le proprie operazioni eventualmente su base di quelle del contesto precedente, con dati aggiornati e senza ridondanze o errori.

Per ogni buffer, si tiene traccia di:

- Una **PRODUCER sync fence**: inserita quando il nodo che elabora un'immagine termina di scrivere i dati;
- Molteplici **CONSUMER sync fences**: inserite al termine della "consumazione" dei dati per marcare una "lettura completata".

Le sync fences, oltre ad un fine di sincronizzazione, agevolano un risparmio della memoria, tramite il **riciclo dello stesso buffer SVUOTATO** al termine di ogni contesto.

Implementazione

Tracer

Il modulo **TRACCIANTE** (o **Tracer**) segue i singoli pacchetti attraverso il grafo e ne registra i timing degli eventi lungo la via.

Con ogni evento, è registrato un **TraceEvent**, una struttura con diversi dati come **event_time**, **packet_timestamp**, **packet_data_id**, **node_id** e **stream_id**. Queste

informazioni nel TraceEvent sono sufficienti per seguire il flusso di dati attraverso il grafo. Il Tracer riporta anche degli **istogrammi** delle varie risorse.

I dati del timing registrati consentono la valutazione e il reporting delle esecuzioni individuali, favorendo la diagnosi di problematiche come delay inaspettati e latenze.

Il modulo Tracer registra le informazioni di timing su richiesta e può essere abilitato tramite una sezione del **GraphConfig**. Analogamente all'abilitazione, può essere anche omesso dall'utente.

Visualizer

Il **VISUALIZER** permette all'utente di comprendere la topologia e il comportamento generale delle pipeline. Nello specifico infatti, verifica:

- La **visuale della TIMELINE**: l'utente può caricare una traccia preregistrata e vedere la tempistica dei pacchetti in movimento attraverso i thread e i calculators;
- La **visuale del GRAFO**: lo user può anche visualizzare la **topologia del grafo**, consentendo di osservare lo stato del grafico in qualsiasi momento, inclusi gli stati di ogni calculator e dei packets elaborati o trattenuti nelle code di input.

Object detection con MediaPipe - Detection + Tracking

Dipendentemente dalla piattaforma dalla quale si voglia svolgere una object detection ad un frame rate medio-alto (ad esempio 30 FPS), è possibile che sia richiesto un elevato consumo di risorse, per alcuni device, difficile da gestire e sopportare.

Per una performance ottimale, con MediaPipe il **tracciamento** e il **rilevamento** **AGISCONO IN PARALLELO**, così che il tracker non sia bloccato dall'azione del detector e viceversa durante la ricezione di ogni frame.

Il grafo dunque si divide in una **Detection branch** e in una **Tracking Branch**:

- La **Detection Branch** seleziona alcuni frame e analizza i **cambi di scena**, dunque i cambiamenti sostanziali tra i frame acquisiti;
- La **Tracking Branch** invece si limita al **calcolo** della differenza dei pixel che si interpongono tra il frame precedente e quello corrente, valutando dunque di quanto si sia mosso un determinato oggetto durante l'intero rilevamento.

Ottenute così le informazioni, agisce il nodo **DETECTION-MERGING NODE**, un nodo di fusione che **unisce le nuove detection al frame originale**, annotandole sul frame stesso.

Fonti : MediaPipe

“Guida al rilevamento delle posizione dei punti di riferimento” :

https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker?hl=it

“Guida alle soluzioni MediaPipe” :

https://ai.google.dev/edge/mediapipe/solutions/guide?hl=it#get_started

“Mediapipe Holistic” :

<https://research.google/blog/mediapipe-holistic-simultaneous-face-hand-and-pose-prediction-on-device/>

“Blazepose: On-device Real-time Body Pose tracking” :

<https://arxiv.org/pdf/2006.10204>

“MobileNetV2: Inverted Residuals and Linear Bottlenecks” :

<https://arxiv.org/pdf/1801.04381>

“Feature Pyramid Networks for Object Detection” :

<https://arxiv.org/pdf/1612.03144>

“Objects as Points” :

<https://arxiv.org/pdf/1904.07850>

“MediaPipe: A Framework for Building Perception Pipelines” :

<https://arxiv.org/pdf/1906.08172>

Librerie implementate e modelli
aggiuntivi

Librerie implementate e modelli aggiuntivi

NumPy (Numerical Python)

La libreria NumPy presenta il pacchetto fondamentale utile al computing scientifico su Python. Prevede infatti un assortimento di routine per operazioni rapide su oggetti derivati come array mascherati o matrici, includendo manipolazioni logiche, matematiche e di forma.

Lavorando su delle acquisizioni video, la libreria NumPy svolge un ruolo cruciale, data l'effettiva applicazione di oggetti di natura **matriciale**.

Tra i metodi NumPy inseriti all'interno dei codici implementati troviamo:

- `np.array( )` : Crea un array.
- `np.squeeze( )` : **Rimuove** da un array multidimensionale le **dimensionalità uguali a 1**, semplificando l'array.
- `np.expand_dims( )` : Al contrario di `np.squeeze( )`, **aggiunge delle dimensionalità 1** all'array.
- `np.multiply( )` : Prodotto scalare.
- `np.dtype( )` : Descrive e stabilisce esattamente **quanti byte** si debba allocare per ogni singolo elemento ed il **tipo** di ciascuno di essi.
- `np.zeros( )` : crea un array e lo riempie di 0.
- `np.hstack( )` : Sta per "**horizontal stack**" e **concatena** due o più array, affiancandoli tra di loro.
(Utilizzato per affiancare al frame di base un frame completamente nero, dove visualizzare accuratamente lo scheletro di rilevamento)
- `np.mean( )` : Media aritmetica.
- `np.abs( )` : Valore assoluto.

OpenCV

OpenCV (Open Source Computer Vision Library) è, come evidenziato dal nome, una libreria open source che presenta al proprio interno centinaia di algoritmi di computer vision.

All'interno della struttura modulare OpenCV sono presenti numerose librerie di image processing, gestione e rilevamento di oggetti, camera calibration etc.

OpenCV va implementato tramite **import cv2**.

All'interno dei codici utilizzati troviamo:

- `cv2.line( )` : Traccia una linea dritta tra due punti.
- `cv2.circle( )` : Costruisce un cerchio (vuoto o pieno).
- `cv2.rectangle( )` : Costruisce un rettangolo.
- `cv2.cvtColor( )` : **Converte il colore** di un immagine passando da una classe (come BGR) ad un'altra (come RGB).
- `cv2.resize( )` : **Ridimensiona** l'immagine, cambiando i pixel in larghezza e altezza.
- `cv2.putText( )` : Applica un testo in sovrapposizione sul frame.
- `cv2.VideoCapture( )` : Forza l'accensione della telecamera del dispositivo o apre un file video salvato nel disco e esplicitato nel codice.
- `cv2.imshow( )` : Apre una finestra del sistema operativo e ne applica l'immagine.
- `cv2.waitKey( )` : Mette **in pausa** il codice per un determinato lasso di tempo **in attesa che non arrivi un input della tastiera**.
- `cv2.destroyAllWindows( )` : Distrugge tutte le finestre create.

Matplotlib - Pyplot

Matplotlib è una libreria volta alla creazione di visualizzazioni statiche, animate e interattive per Python.

Pyplot è un'interfaccia basata sullo stato di matplotlib, che consente **implicitamente** di tracciare, aprire le figure e gestirne le **GUI** (interfacce grafiche utente).

L'uso di pyplot è inteso generalmente per **plots interattivi** e casi semplici di **plot**.

Tra le funzioni impiegate, troviamo funzioni per la creazione di grafici (supponendo di importare pyplot come "plt" come da convenzione):

- `plt.subplots( )` : **Aggiunge un asse** ad una figura corrente o ne recupera uno esistente.
- `plt.suptitle( )` : Aggiunge un sottotitolo alla figura.
- `plt.plot( )` : **Traccia y rispetto ad x** come linee e marker.
- `plt.set_title( )` : Setta un titolo della pagina plottata.
- `plt.legend( )` : Struttura una legenda.
- `plt.tight_layout( )` : **Regola le spaziature tra i sottografici ed attorno ad essi**.
- `plt.savefig( )` : Salva la figura corrente.
- `plt.show( )` : Mostra tutte le figure.
- `plt.ylabel( )` : Imposta un'**etichetta** per l'asse y. Analogico con `xlabel( )`.
- `plt.xticks( )` : Ottiene ed importa le posizioni dei tick e le etichette dell'asse x. Analogico con `yticks( )`.

- `plt.tick_params( )` : Personalizza l'aspetto dei tick, le tacche di riferimento, modificando dimensioni, colore, posizione etc.

Psutil

La libreria psutil sta per **Python System and Process Utilities** si occupa del recupero di informazioni sui processi in esecuzione e sull'utilizzo del sistema (quindi CPU, memoria, dischi, rete, sensori, etc.). Presenta la sua utilità nel monitoraggio del sistema e nella gestione dei processi in esecuzione.

(Utilizzato per monitorare l'uso di CPU e RAM durante la detection)

Le funzioni utili ai fini del progetto:

- `.Process( )` : Ricevuto un **pid (PROCESS ID)** crea un collegamento con esso, **ignorando gli altri eventuali processi in esecuzione**.
Richiamando la libreria **os**, con `.Process(os.getpid( ))` è possibile creare un collegamento diretto con il SISTEMA OPERATIVO, ignorando processi secondari nel monitoraggio performativo della detection.
- `.cpu_percent( )` : Calcola la **percentuale della CPU in uso** in un preciso istante.
- `.memory_info( )` : Il metodo `memory_info( )` si informa della quantità di memoria impiegata dal processo.
Nel programma la funzione è integrata con `.rss`, che chiede il **Resident Set Size**, ovvero la quantità di **RAM** interessata nello specifico istante.

OS

Il modulo **os** consente l'uso di funzionalità dipendenti dal sistema operativo.

Tra le funzioni impiegate:

- `os.getpid( )` : Ritorna il corrente **process id**.
- `os.path.basename( )` : **Estrae** il nome di un file, per il quale percorrere un path, lasciando solo il nome del file e omettendo le cartelle precedenti.

Glob

Il modulo glob trova i pathnames utilizzando regole di corrispondenza dei modelli simili alla shell UNIX. Concretamente glob è volto alla **ricerca di file e cartelle** all'interno del device sfruttando delle **WILDCARDS**, dei caratteri jolly come **(*)** (che indica qualsiasi lettera).

Unica funzione della libreria glob impiegata all'interno dei codici:

- `glob.glob( )` : Restituisce una Lista contenente **i datapath corrispondenti ad un nome o pattern dato**.
Il fatto che i risultati siano restituiti alfabeticamente o meno, dipende dal sistema operativo stesso.

Pandas

Pandas è una libreria utilizzata per lavorare con dei data-set, con funzioni di analisi, pulizia, esplorazione e manipolazione dei dati.

Tra i vantaggi caratteristici di pandas:

- Gestione dei dati mancanti (NaN) in dati a virgola mobile e non;
- Dimensioni mutabili;
- **Allineamento specifico** dei dati tramite un set di etichette o **allineamento automatico**;
- Conversione semplificata di dati irregolari;
- **Rimodellamento flessibile e rotazione dei set di dati**.

Tra le funzioni impiegate all'interno dei codici:

- `.DataFrame( )` : Si tratta di una funzione che **converte** i dati in un vero e proprio DataFrame, una griglia con righe e colonne pronta per l'analisi.
- `.to_excel( )` : Prende il DataFrame e lo salva come **file .xlsx** all'interno del disco.
- `.describe( )` : Genera istantaneamente un **riepilogo statistico** delle informazioni accumulate.
- `.groupby( )` : Raggruppa dei dati **per categoria**.

Time

Il modulo time fornisce funzioni legate al tempo.

Sfruttiamo la funzione `time.perf_counter( )` per ritornare un valore di un **performance counter** (contatore prestazionale), un contatore nei termini dei millesimi di secondo, che inizia dalla chiamata stessa della funzione.

All'interno dei notebook è impiegato per il calcolo degli **fps**:

- Si dichiara prima **prev_time = 0**;
- Successivamente **curr_time = time.perf_counter()** inizia a contare i secondi affinché il calcolo degli FPS si ottenga con il rapporto $\frac{1}{(currTime - prevTime)}$;
- Dopo il primo calcolo il **prev_time** sarà aggiornato al **curr_time** affinché prosegua il ciclo.

Pytorch

Pytorch conferisce due feature principali ad alto livello:

- **Calcolo tensoriale** come NumPy con un'elevata **accelerazione della GPU**;
- Reti neurali profonde costruite su un sistema **AUTOGRAD** (un motore di DIFFERENZIAZIONE volto a calcolare gradienti nelle reti neurali) a nastro.

Tra le componenti di Pytorch:

- **torch** : Una libreria sui tensori come NumPy, con un forte supporto per le GPU;
- **torch.autograd** : Libreria di differenziazione che supporta le operazioni sui tensori;
- **torch.jit** : Uno stack di compilazione per creare modelli ottimizzabili dal codice Pytorch.
- **torch.nn** : Una libreria di reti neurali profondamente integrate con un autograd pensato per garantire una maggiore flessibilità.
- **torch.multiprocessing** : Multiprocessing di Python con una condivisione della memoria dei tensori tra i processi.
- **torch.utils** : Fornisce il DataLoader e altre utility functions.
(Il DataLoader è una classe built-in che fornisce metodi efficienti di caricamento dei dati).

Generalmente Pytorch è impiegato come sostituto di NumPy o come suo supporto per accelerare la GPU durante il suo impiego.

Pytorch costruisce le reti neurali utilizzando e riproducendo un registratore. Con Pytorch si sfrutta la tecnica dell'**autodifferenziazione inversa** che consente di **modificare il comportamento della rete** in maniera arbitraria, senza rallentamenti o sovraccarichi.

Pytorch è stato impiegato esclusivamente nei blocchi di codice dedicati al programma ibrido MediaPipe + YOLO_v8 e con poche funzioni:

- **torch.backends.mps.is_available()** : Una funzione che svolge una diagnosi per verificare se l'architettura **MPS (Metal Performance Shaders)** sia o meno disponibile e supportata, affinché i dispositivi MacOS impiegati potessero sfruttare i vantaggi del chip Apple Silicon (nel nostro caso **M1** e **M3**).
Si tratta nell'effettivo di una funzione che stabilisce se assegnare il lavoro di YOLO alla GPU "**mps**" oppure alla CPU.
- **.cpu()** : Calcola i risultati sulla scheda video sottoforma di Tensori Pytorch e li **sposta fisicamente dalla memoria della GPU fino alla RAM**.

YOLO

Il modello **YOLO (You Only Look Once)** è un modello di **object detection** e segmentazione delle immagini, in continuo sviluppo e aggiornamento dal 2015.

La famiglia di modelli YOLO si trova al centro dello sviluppo e dell'evoluzione dell'object detection, per via del suo approccio all'uso delle **BOUNDING BOXES** e l'implementazione di funzioni di **predizione** e valutazione delle probabilità sulle immagini.

La versione utilizzata all'interno del modello integrato insieme a MediaPipe è **YOLOv8**, rilasciata da Ultralytics nel 2023.

Alla sua uscita prevedeva infatti, oltre a delle migliorie sul lato performativo, un incremento della flessibilità di utilizzo e un comparto ricco di **tasks AI legate alla computer vision**.

YOLOv8

La serie YOLOv8 offre una serie di modelli, ciascuno specializzato in diversi compiti di computer vision:

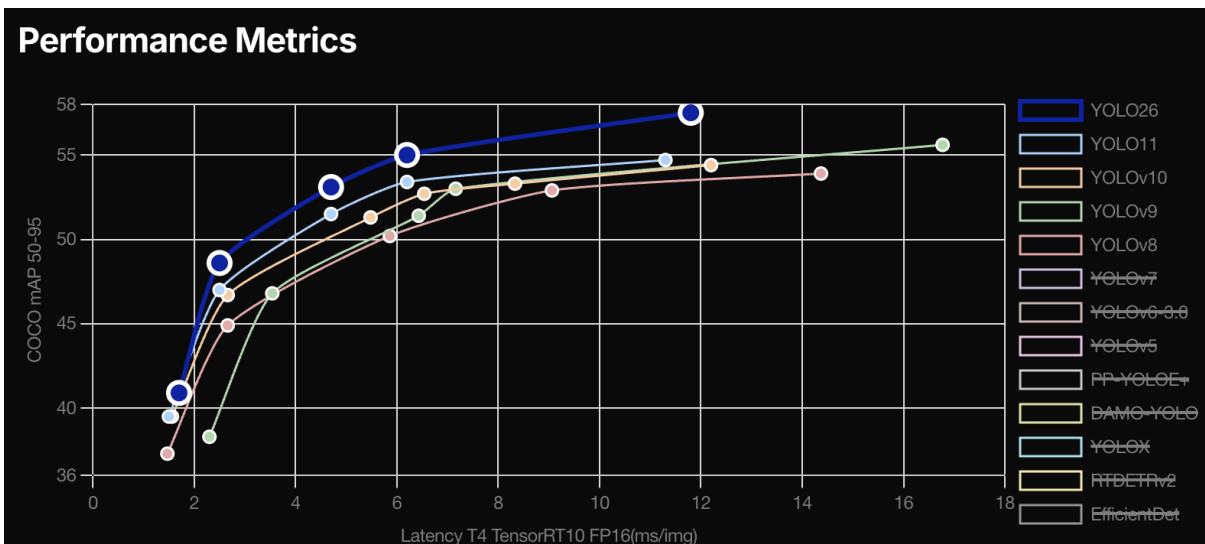
- **YOLOv8** : Object detection alla base della serie di modelli;
- **YOLOv8-seg** : Segmentazione dell'istanza;
- **YOLOv8-pose** : Creazione delle pose e ricezione dei Keypoints
- **YOLOv8-obbb** : Detection orientata;
- **YOLOv8-cls** : Classificazione.

Metriche di performance

Si considerano generalmente quattro valori chiave per indicare le performances di un modello di object detection come YOLO:

- **mAP**, ovvero **Mean Average Precision**, una metrica circa l'**ACCURATEZZA** nel riconoscimento di diverse classi;
- **Inference time**, ovvero la **VELOCITÀ** del modello nel **processare le immagini**;
- **Training Time**, ovvero l'**EFFICIENZA** del processo di addestramento;
- **Model Size**, ovvero la **RICHIESTA COMPUTAZIONALE** per il funzionamento del modello (minor richiesta implica migliori performances).

Per ragioni implementative, YOLOv8 risulta, ai fini del lavoro svolto, un compromesso adatto tra performance e accuratezza. Rispetto infatti ai suoi modelli direttamente successivi:



Nel grafico è possibile verificare come, escluso YOLOv26, la versione più recente, i suoi predecessori non presentino marcate differenze in termini di latenza e di performance di accuratezza (in metrica **COCO mAP 50-95** - Mean Average Precision, nell'ordine da 0.5 a 0.95).

Modelli di YOLOv8

All'interno del programma, per la precisione, è stato richiamato il modello **YOLOv8n**, ossia il modello “**Nano**”, la versione più leggera e compatta della serie.

Infatti tutti i modelli e le versioni di YOLO presentano una distinzione a seconda della dimensione:

- **Nano (n)**;
- **Small (s)**;
- **Medium (m)**;
- **Large (l)**;
- **Extra-Large (x)**.

Ovviamente maggiore è la dimensionalità della variante e **superiore sarà la sua accuratezza**, a discapito però di **requisiti di latenza proporzionalmente maggiori**.

Poiché YOLO è stato esclusivamente impiegato per l'utilizzo delle BOUNDING BOXES e non per la sua componente di Pose Estimation, è stata ritenuta non necessaria l'implementazione di versioni di dimensione superiore.

Inoltre è da considerare che la realizzazione del progetto sia stata svolta su **CPU**, **senza l'ausilio di GPU dedicate**.

Più dettagliatamente, è possibile descrivere i cinque modelli dell'architettura v8:

- **YOLOv8n** : Il modello più **leggero** e **rapido** della serie v8. Con risorse più limitate rispetto alle sue versioni più avanzate, gode della compattezza a discapito di un **minor numero di parametri riconoscitivi**;
- **YOLOv8s** : Si considera come il modello BASE della serie e presenta circa **9 milioni** di parametri. Rispetto al precedente introduce una migliore feature

fusion e una detection accuracy maggiore, soprattutto nei confronti di oggetti di piccole dimensioni;

- **YOLOv8m** : Caratterizzato da circa **25 milioni** di parametri, presenta un'architettura più costosa, con dei BACKBONE e NECK più profondi a discapito dell'efficienza computazionale più ridotta;
- **YOLOv8l** : Caratterizzato da circa **55 milioni** di parametri, risulta un'esclusiva utile solo per applicazioni con una richiesta di precisione **molto elevata**. Comprende dei **layer addizionali** per un'estrazione delle feature più definita e una più affinata capacità di ricerca di oggetti di dimensioni sempre minori;
- **YOLOv8x** : Il modello più grande della serie, con circa **90 milioni** di parametri e la maggiore **mAP** (mean Average Precision). A causa di un costo computazionale a sé molto elevato, richiede necessariamente il supporto di GPU High-End.

Model	Params (Million)	Accuracy (mAP@0.5)	CPU Time (ms)	GPU Time (ms)
YOLOv8n	2.0	47.2	42	5.8
YOLOv8s	9.0	58.5	90	6.0
YOLOv8m	25.0	66.3	210	7.8
YOLOv8l	55.0	69.8	400	9.8
YOLOv8x	90.0	71.5	720	11.5

Architettura

L'architettura di YOLOv8 è strutturata su tre componenti core:

- **BACKBONE**: Sfrutta una rete neurale CONVOLUZIONALE che cattura le immagini in input estraendo le features e assicurando una object detection accurata.
La backbone comprende un modulo **CSP BOTTLENECK**, ossia un modulo **incrociato stadio parziale a collo di bottiglia**, con lo scopo di **RIDURRE** una **ridondanza computazionale** a favore invece di un **riutilizzo** delle feature. Questo modulo si completa con l'integrazione del **FPN (Feature Pyramid Network)** per l'aggregazione efficiente di oggetti in multi-scala.
- **NECK**: Rifinisce e fonde le features accumulate dalla Backbone su **differenti scale**, per poter rilevare oggetti di piccole o grandi dimensioni. Il Neck utilizza una versione ottimizzata del **PANet (Path Aggregation Network** - architettura multi-scale volta a ricevere una **buona risoluzione spaziale** della piramide delle feature su scale diverse).
- **HEAD**: Modulo responsabile della generazione delle predizioni finali, tra cui le **coordinate delle bounding boxes**, dei **confidence score**, dei livelli etc.
Il modello v8 introduce un approccio **ANCHOR-FREE**.

(Nella object detection, si intende come "anchor" una bounding box **predefinita**, con dimensioni e proporzioni fisse. Nel caso di YOLOv8 e dei suoi successori, la

presenza di un'ancora dunque **viene omessa**. Il modello **predice direttamente le coordinate della bounding box SENZA PARTIRE DA BOX PREDEFINITE**.

Metodologie

Object detection ad alta performance di YOLOv8 presenta alcune metodologie di **training**, volte all'incremento prestazionale del modello:

- **Data Augmentation avanzato**: Sono implementate numerose strategie per il miglioramento della **generalizzazione del modello**, come la COMBINAZIONE di immagini multiple all'interno di un **singolo blocco di allenamento**, tramite una tecnica di "**MOSAIC AND MIXUP AUGMENTATION**", che **combina** quattro o più immagini distinte in un **singolo esempio di allenamento**, esponendo il modello a più scale di oggetti, posizioni e configurazioni spaziali.

(Generalmente intendiamo come Data Augmentation l'aumento artificiale della quantità e diversità dei dati per favorire un addestramento efficace delle reti neurali)

- **Funzione di Perdita Focale**: È presente una funzione di "**focal loss**" per azioni di CLASSIFICAZIONE, utile al riconoscimento di oggetti molto piccoli oppure offuscati (o parzialmente coperti).
- **Transizione a PyTorch per ottimizzare**: Implementando e sfruttando delle transizioni con PyTorch, YOLOv8 ottimizza la propria velocità di allenamento e i propri inference time (quantità computazionale richiesta per lo svolgimento delle previsioni) mantenendo la propria accuratezza.

Calcolo della Perdita

Definiamo come **Loss** la perdita che il modello subisce in caso di errore durante il training.

Si considerano:

- **Focal Loss Function**: **Omette** le percentuali di penalità su casi di riconoscimento **ben noti**, ottimizzando il processo di CLASSIFICAZIONE degli oggetti rilevati.
- **IoU Loss**: IoU sta per **INTERSECTION OVER UNION**, una formula che calcola la **sovrapposizione** tra il box rappresentato e quello ideale per l'oggetto considerato. Questa Loss dunque allena il modello nella PREVISIONE della dimensione e del posizionamento corretto e preciso della bounding box.

- **Objectness Loss:** Si assicura che il modello si concentri su zone di rilevazione **dove sia rilevabile un oggetto**.

Addestramento a precisione mista

YOLOv8 utilizza un training a **precisione mista**, una tecnica che permette al modello il leveraggio di **16 bit floating-point di precisione** durante le previsioni. Questo genere di tecnica trova maggiore utilità per implementazioni su GPU compatibili. Ciononostante presenta altre utili migliorie come una riduzione del consumo di memoria da parte del modello.

Analisi dei codici

Analisi dei codici

Struttura Comune dei Notebook

Tutti i modelli condividono una **struttura a pipeline**, ogni **frame video attraversa una sequenza fissa di funzioni indipendenti**, chiamate in ordine dal ciclo **Main principale**. Tutti i modelli condividono una struttura a pipeline, ogni **frame video attraversa una sequenza fissa di funzioni indipendenti**, chiamate in ordine dal ciclo **Main principale**. Tuttavia, per garantire la **stabilità del software**, è stato necessario configurare **tre ambienti virtuali distinti attraverso l'utilizzo di conda**, a causa di incompatibilità tra le dipendenze.

L'ecosistema **TensorFlow** (utilizzato per far girare Lightning, MultiPose e Thunder) non può coesistere stabilmente nello stesso ambiente di **MediaPipe**.

Frame e video processing.

Un **frame** è una singola immagine estratta da un flusso video. OpenCV legge i video come sequenze di **frame tramite cv2.VideoCapture()**, restituendo ad ogni iterazione del ciclo **while cap.isOpened()** un frame grezzo come array NumPy tridimensionale di forma **[altezza, larghezza, 3]**, dove i 3 canali rappresentano i colori in formato **BGR (Blue-Green-Red)** convenzione di default di OpenCV diversa dall'RGB standard.

Per ciascun frame vengono eseguiti cinque stadi in sequenza:

- **Stage 1 - Pre-processing:** preparazione del frame per il modello (resize, cambio spazio colore, cast del tensore)
- **Stage 2 - Inferenza:** passaggio del tensore al modello per ottenere i keypoint
- **Stage 3 - Post-processing:** calcolo delle metriche di qualità (confidenza, keypoint rilevati, jitter)
- **Stage 4 - Rendering:** visualizzazione dello scheletro sul frame originale e su sfondo nero, affiancati con **np.hstack()**
- **Stage 5 - Raccolta metriche:** salvataggio di tutte le misure in un dizionario che alimenta il **DataFrame finale esportato in Excel**.

Ogni video viene eseguito **due volte (run 1 e run 2)** per analizzare la stabilità. Un modello deterministico e ben ottimizzato produce metriche simili tra le due esecuzioni sullo stesso contenuto.

Inoltre all'inizio di ogni modello abbiamo incluso le importazioni e dipendenze, ma anche una serie di **print**, affinché vengano esplicitati la versione di TensorFlow o Mediapipe utilizzata, GPU, CPU e RAM utilizzata.

Metriche comuni in tutti i modelli.

Le metriche utilizzate in tutti i modelli sono:

- **FPS (Frames Per Second):** rappresentano il principale indicatore di reattività. La metrica viene calcolata dinamicamente misurando l'**intervallo temporale tra due frame consecutivi** tramite `time.perf_counter()`:
$$\frac{1}{(curr_{time} - prev_{time})}$$
- **CPU (%) e RAM (MB)** : percentuale di utilizzo di CPU usata dal processo e memoria occupata dal processo (espressa in MB). Questi dati vengono estratti tramite la **libreria psutil**, analizzando nello specifico il **Resident Set Size (rss)** del processo per avere una misura reale della memoria fisica occupata. Per calcolare utilizziamo: `psutil.cpu_percent()` e `process.memory_info().rss/(1024*1024)`, il quale restituisce un valore della memoria in byte, per passare da un'unità all'altra ci muoviamo a passi di 1024, quindi 1 MB è dato da 1024 x 1024 byte.
- **Confidence (confidenza media):** questa metrica rappresenta il grado di certezza del modello rispetto ai keypoint rilevati. Viene calcolata come la **media degli score di visibilità di tutti i punti** che superano una determinata soglia di confidenza. Una **confidenza alta** indica che il modello sta operando in **condizioni ottimali di luce e visibilità del soggetto**.
Viene calcolata quindi come:
$$\frac{media\ degli\ score}{visibility\ su\ tutti\ i\ keypoints\ sopra\ la\ soglia}$$
- **Jitter (stabilità del tracking)** : Il jitter misura l'**instabilità** o il tremolio del tracking tra frame successivi. Viene quantificato calcolando la **distanza media assoluta delle coordinate dei punti** tra il **frame attuale (t)** e quello **precedente (t-1)**. Un **valore di jitter basso** è sinonimo di un **tracking fluido e privo di micro-scatti**.

Raccolta delle Metriche e Export

In tutti i modelli la funzione `collect_metrics()` produce ad ogni frame un **dizionario** con nome del video, numero esecuzione, numero frame, utilizzo CPU e RAM, FPS, confidenza e jitter. Questi dizionari vengono accumulati nella lista `project_data` e al termine convertiti in un **DataFrame Pandas** con `pd.DataFrame(project_data)`.

Il file Excel viene salvato con nome che include il nome del dispositivo `platform.node()` per distinguere automaticamente i risultati ottenuti sul MacBook M1 da quelli su MacBook M3.

Analisi della stabilità tra i due run

La cella di analisi della stabilità raggruppa il **DataFrame per numero di esecuzione** con `df.groupby('Execution')` e calcola le medie di **FPS, CPU, RAM**, confidenza e jitter.

Il delta tra **run 2 e run 1 (summary.loc[2] - summary.loc[1])** quantifica la variabilità. I valori prossimi a zero indicano un comportamento deterministico e stabile del modello, mentre delta elevati segnalano dipendenze dal warm-up del sistema operativo o dalla cache della CPU.

Grafici di sintesi

Vengono infine generati due grafici:

- Il primo, **metrics_mean_overview.png**, mostra la **media di ciascuna metrica per ogni video tramite grafici a barre**, permettendo di identificare video che mettono in **difficoltà il modello**, quindi con **FPS bassi e confidenza ridotta**;
- Il secondo, **run_comparison.png**, confronta le medie del **run 1 e run 2** per tutte le metriche, offrendo una **visualizzazione immediata della stabilità del modello tra esecuzioni successive**.

MoveNet Lightning - SinglePose

Il primo modello in ordine di sviluppo all'interno del progetto, prevede l'implementazione della variante **Lightning** di **MoveNet**, tramite l'interprete **TensorFlow Lite**.

A differenza dei due modelli successivi, dedicati a MoveNet MultiPose e MoveNet Thunder, il seguente modello è l'unico dei tre la cui base TensorFlow è stata implementata tramite file (**3.tflite**). Le sue controparti invece integrano TensorFlow direttamente dalla Hub online.

Dizionario EDGES

Successivamente all'insieme di **import** delle librerie necessarie, è presente un **DIZIONARIO "EDGES"** che mappa ogni COPPIA di keypoint con un **colore distintivo**:

- "m" : magenta;
- "c" : cyan;
- "y" : yellow.

Con l'uso di EDGES vengono definite le **18 connessioni dello scheletro** secondo la topologia ufficiale di MoveNet.

Il dizionario EDGES verrà riproposto tale e quale anche nei modelli successivi della famiglia MoveNet.

Fase di PRE-processing

La funzione **pre_processing(frame)** prepara il frame grezzo tramite tre passaggi principali:

1. Poiché OpenCV legge in **BGR** e il modello necessita un ritorno in **RGB**, inizialmente avviene una **conversione** dello spazio colore da **BGR** a **RGB** tramite il metodo **cv2.cvtColor()**;
2. Il frame subisce un **ridimensionamento** con **cv2.resize()**, portandosi ad una risoluzione specifica **192 x 192** come previsto dalle caratteristiche default di MoveNet Lightning;
3. Si svolge una **conversione di tipo** tramite un **cast** e si aggiungono una dimensionalità di **1** rappresentante la batch, tramite il metodo **tf.cast(np.expand_dims(img, axis = 0), dtype = tf.float32)**. In questa maniera il tensore ricevuto mantiene la forma **[1, 192, 192, 3]**, ossia:
 - 1 : batch size;
 - 192, 192 : dimensione specifica;
 - 3 : canali di colore (RGB).

Inferenza

La funzione **make_detections(interpreter, input_image)** sfrutta l'interpreter **tf.lite** per recuperare gli **input_details** e gli **output_details**.

Tramite lo stesso interpreter si effettuano dei rilevamenti e si **carica il tensore di input** tramite la funzione:

- **set_tensor(input_details[0]['index'], np.array(input_image))** : La **input_image**, viene **trasformata in un array** e **inserita** nell'indirizzo specifico di **input_details**.
- La funzione **invoke()** AVVIA definitivamente l'interpreter che, **a dati ricevuti**
- Al termine dell'azione di **invoke()** viene chiamata la funzione **get_tensor()** per ricevere le informazioni necessarie per i **keypoints**, i quali verranno ritornati da **make_detections**.

Fase di POST-processing

La funzione **post_processing(keypoints_with_scores, prev_keypoints_with_scores = None, confidence_threshold=0.3)**

è coinvolta nella **traduzione** dei dati grezzi accumulati in effettive statistiche:

- Con **np.squeeze()** la matrice di dati della forma, **rimuove** tutti i valori **"1"** non essenziali, i quali possano considerarsi sottintesi.

Sono calcolate tre metriche:

- **n_detected**, i **keypoints** che rispettano il criterio di confidence **superiore alla soglia prestabilita** (0.3 nel caso corrente);
- **conf_mean**, la confidenza media;
- **jitter**, il valore che determina il CAMBIAMENTO delle coordinate frame per frame.

Rendering

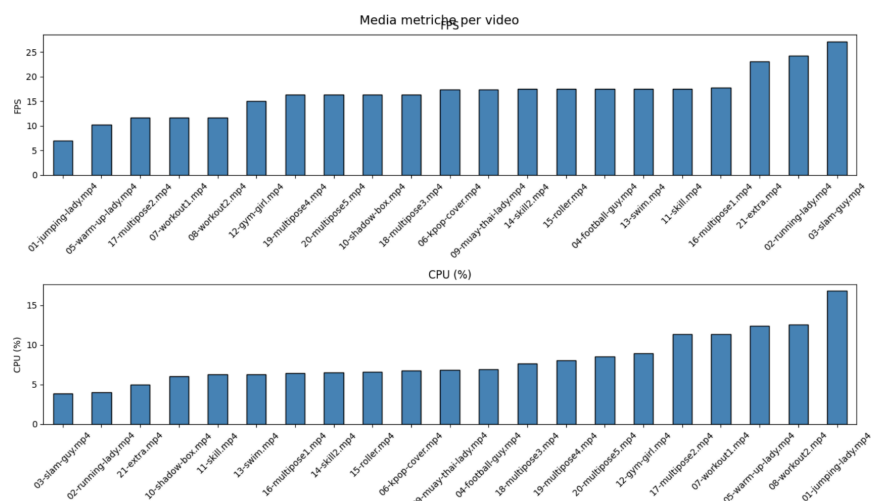
Due funzioni sono alla base del rendering dello scheletro generato dal programma per stimare la posa:

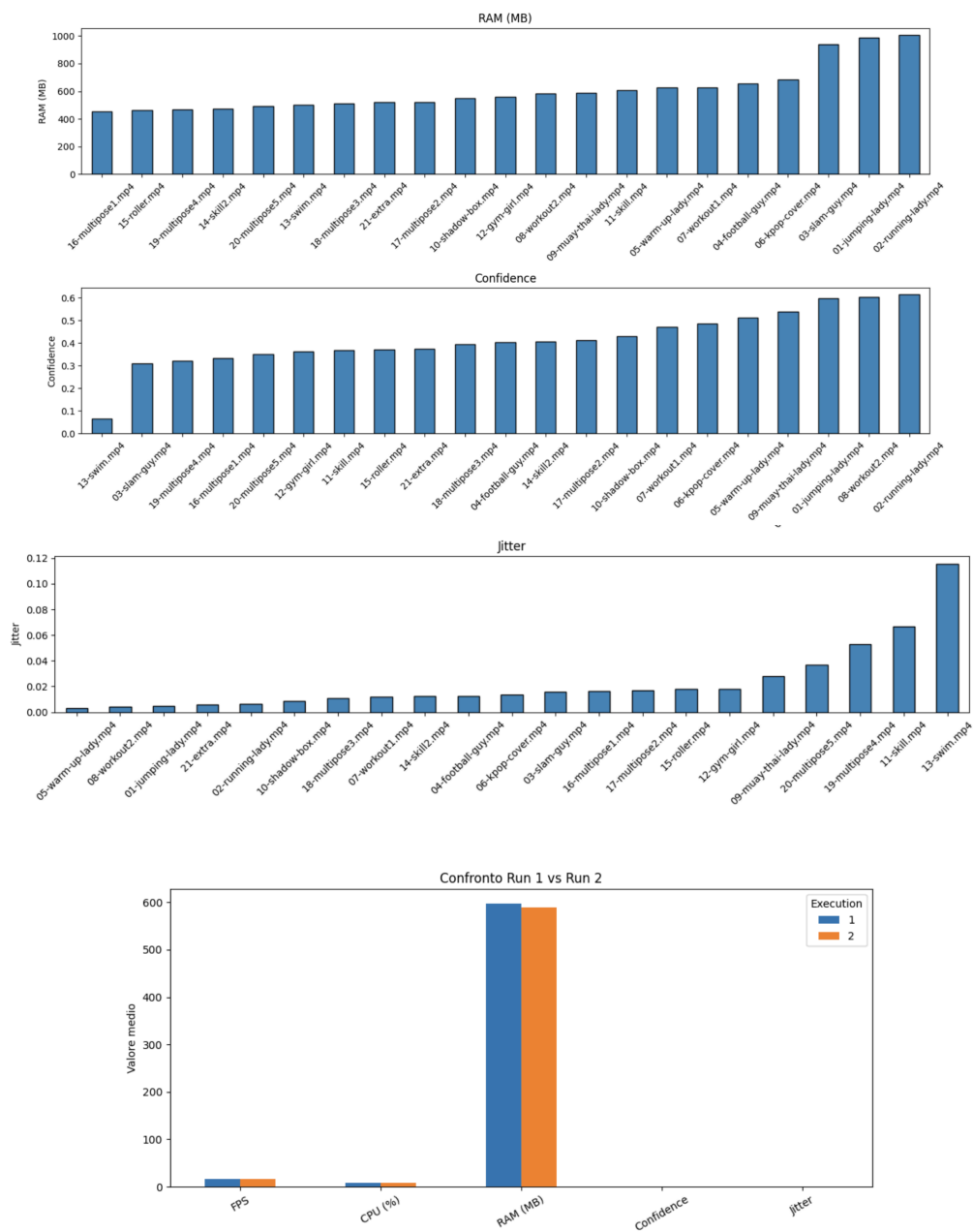
- **draw_keypoints()**: Funzione che **traccia** i 17 keypoints del corpo nel caso in cui essi superino la soglia di confidence prestabilita. Quando è stabilita la possibilità di tracciare il punto, la funzione sfrutta **cv2.circle()** per stampare il cerchio che lo rappresenti.
- **draw_connections()**: Viene impiegato il dizionario EDGES per determinare su quali estremi verificare e nel caso in cui entrambi siano visibili al modello, **viene tracciata una linea che li colleghi**.

Affinché lo scheletro di stima della posa possa essere comparato dal video originale, è stata sviluppata una funzione che ne **crea una replica** su uno sfondo nero **delle stesse dimensioni del frame originale**, **AFFIANCANDO I DUE FRAME ORIZZONTALMENTE** tramite la funzione NumPy `np.hstack()`.

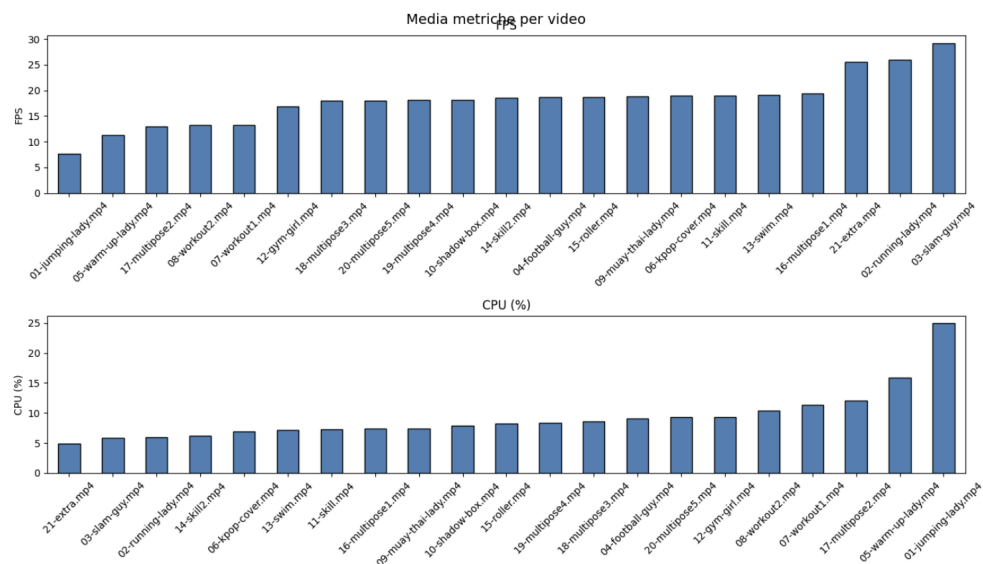
Comparazione prestazioni tra M1 e M3

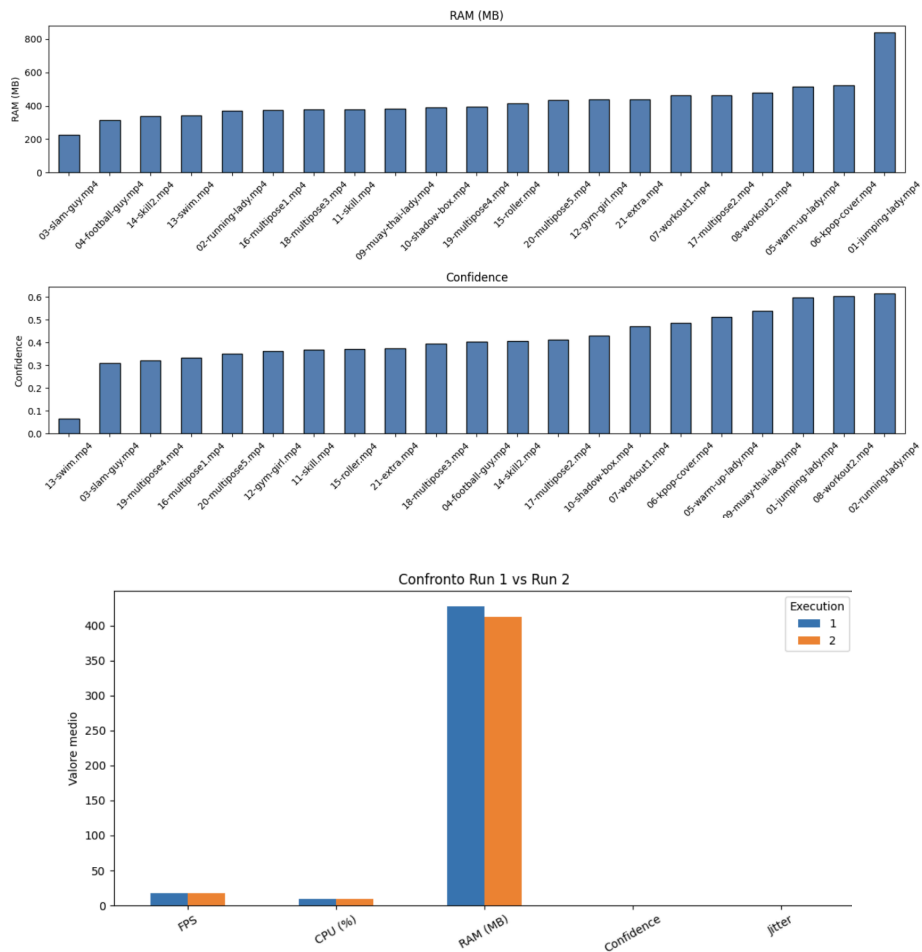
Grafici M1





Grafici M3





Metrica	MacBook Air M1	MacBook Air M3	Differenze
FPS (Frame per Second)	~14.66	~17.86	L'M3 mostra un vantaggio netto in termini di velocità. L'M1 fatica a mantenere 15 FPS, l'M3 raggiunge picchi di 26-30 FPS in alcune sequenze.
RAM (Utilizzo medio)	~614.67 MB	~420.04 MB	L'M3 consuma circa 200MB in meno rispetto all'M1 durante l'esecuzione, questo dimostra un'ottimizzazione superiore per il runtime TensorFlow Lite per M3.

CPU (Utilizzo medio)	~9.10%	~9.13%	L'utilizzo della CPU in M1 e M3 è pressoché identica
Confidence (Fiducia)	0.23	0.41	L'M3 è anche più preciso, il livello di confidenza è quasi raddoppiato
Keypoints rilevati	5.47	12.04	L'M3 rileva in media 12 punti chiave su 17, contro i soli 5.5 dell'M1. Questo significa che l'M3 riesce a tracciare meglio l'intera figura e l'M1 perde spesso il tracking
Jitter (Instabilità)	0.058	0.025	Il jitter è drasticamente inferiore sull'M3, il che si traduce che in un movimento dei punti chiave fluidi e senza scatti improvvisi .

MoveNet Thunder - SinglePose

Il secondo modello in ordine di sviluppo all'interno del progetto, prevede l'implementazione della variante **Thunder** di **MoveNet**, configurata per l'esecuzione in locale tramite i file estratti dall'archivio. L'implementazione sfrutta l'ambiente di TensorFlow, offrendo maggiore flessibilità operativa a fronte di un overhead di avvio maggiore (ovvero il tempo e le risorse che il computer utilizza solo per prepararsi a lavorare). La struttura della pipeline è analoga a quella **Lightning**, con differenze significative nel caricamento del modello, nella risoluzione di input e nella soglia di confidenza.

Parametro	Lightning	Thunder
Risoluzione input	192 x 192 px	256x256px
Tipo tensore in pre-processing	float32	int32

Soglia confidenza	0.3	0.4
Caricamento modello	File 3.tflite locale	TF Hub/Kaggle
Accesso output	interpreter.get_tensor()	results['output_0'].numpy()

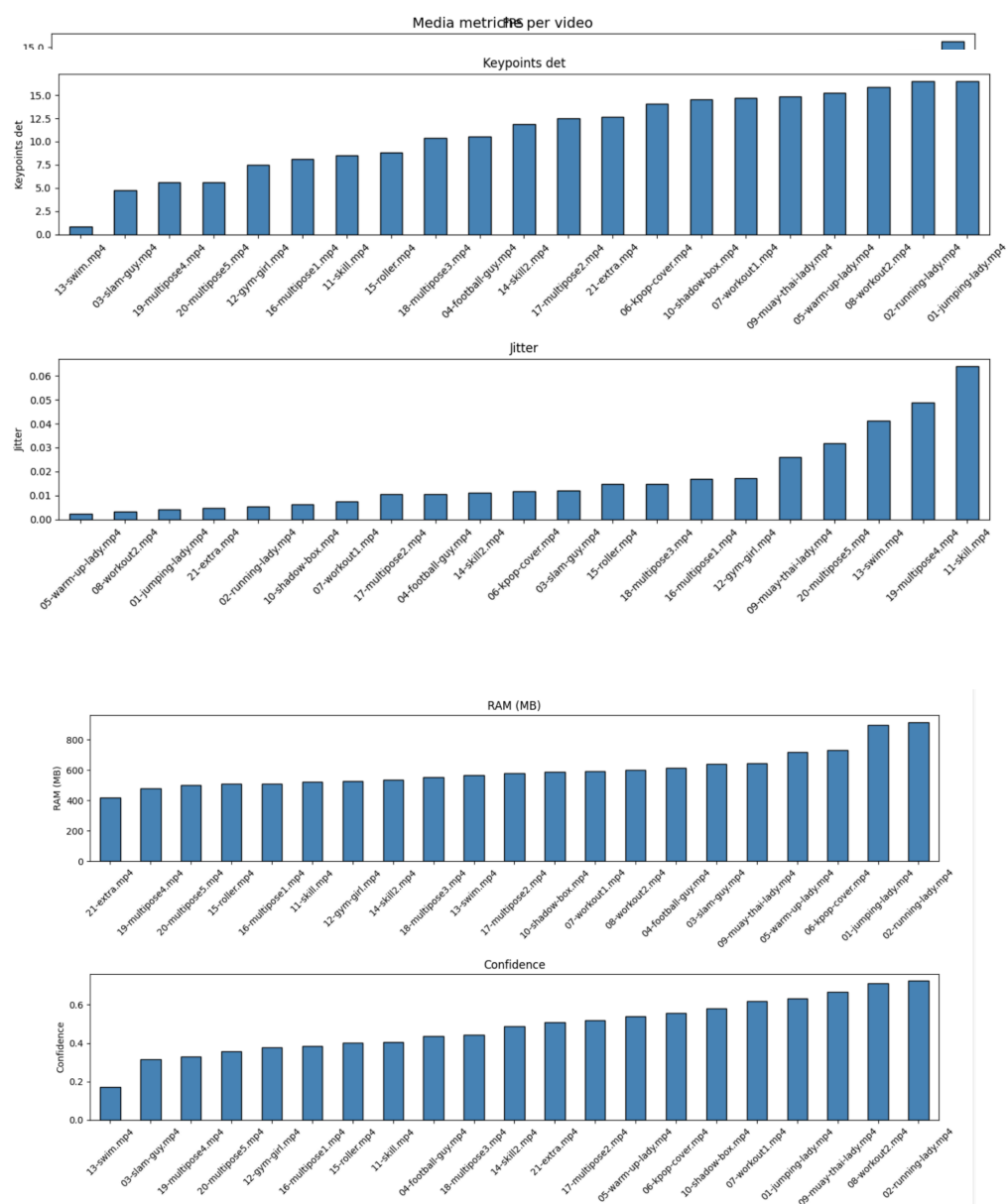
La risoluzione maggiore (256x256) permette a Thunder di catturare dettagli articolari più fini, a scapito di un **costo computazionale superiore per frame**.

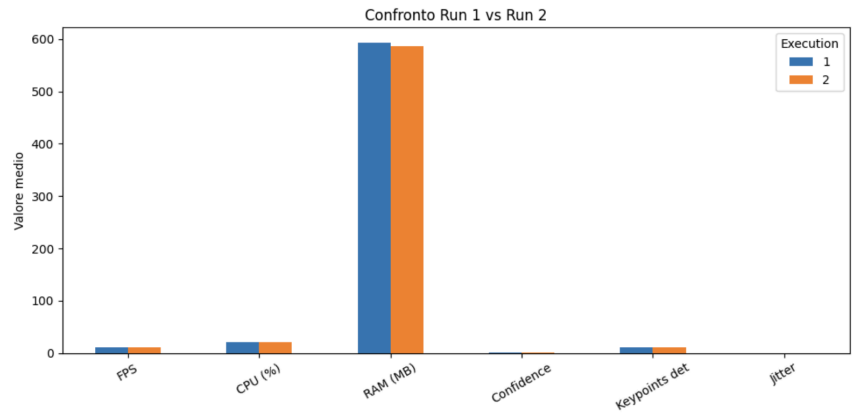
Il cast a **int32** invece di float32 riflette il **formato nativo di input del modello TF completo, che normalizza internamente i valori pixel**.

L'output **results['output_0'].numpy()** restituisce un tensore di forma [1,1,17,3] identico a Lightning. La **soglia di confidenza** è impostata a **0.4 anziché 0.3**, coerentemente con la **maggiore precisione attesa dal modello** che produce score più affidabili.

Comparazione prestazioni tra M1 e M3

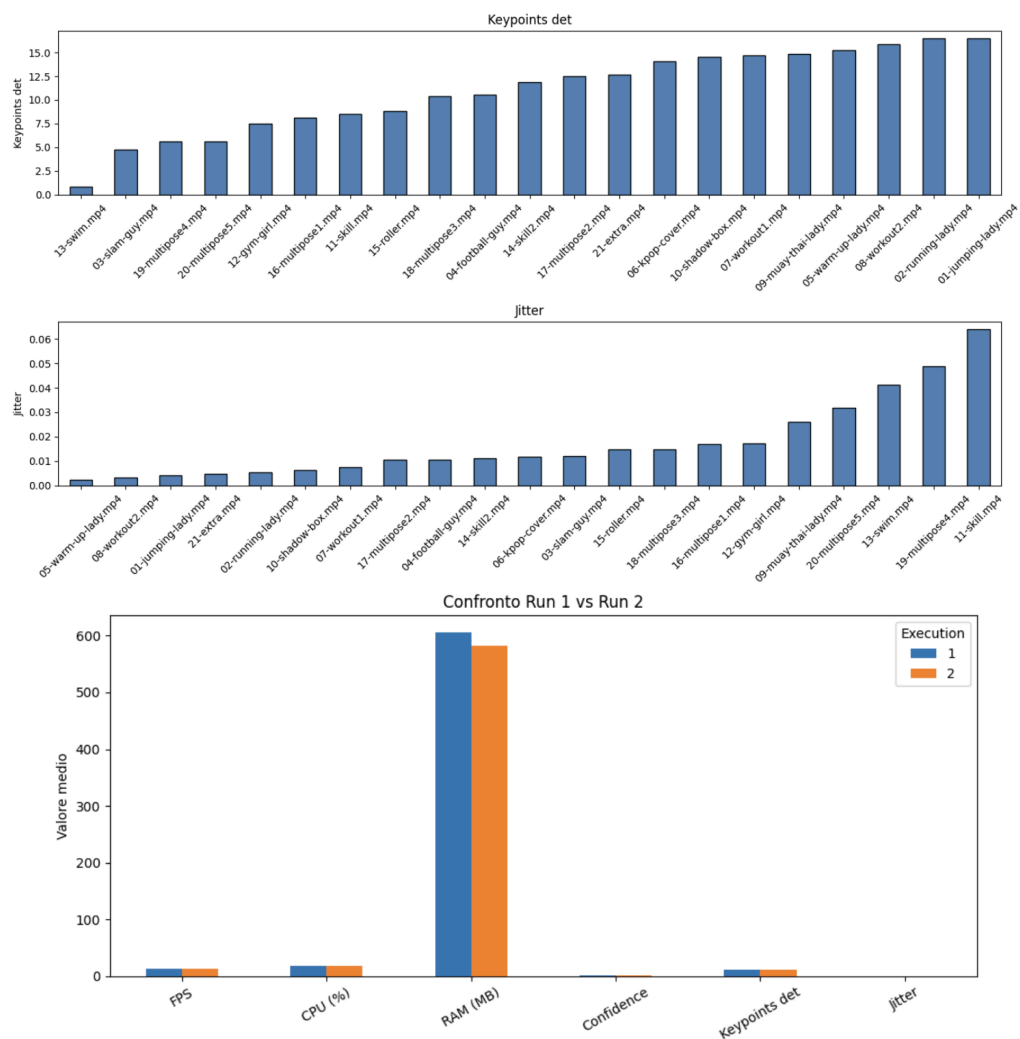
Grafici M1





Grafici M3





Metrica	MacBook Air M1	MacBook Air M3	Differenze
FPS (Frame per Second)	~7.25	~10.51	L'M1 scende rispetto a Lightning a circa 7 FPS e l'M3 riesce a mantenere fluidità superiore a 10.5 FPS.
RAM (Utilizzo medio)	~640MB	~336 MB	L'M3 riduce ulteriormente l'uso della RAM e l'M1 invece sale di 640 MB.
CPU (Utilizzo medio)	~22.3%	~14.8%	Nell'M1 vi sono picchi di carico CPU che superano il 20% e l'M3 utilizza meno la CPU, i Neural Engine dell'M3

			gestiscono i layer convoluzionali più pesanti di Thunder con più disinvoltura.
Confidence (Fiducia)	0.28	0.56	L'M3 mantiene la confidenza molto alta e M1 scende a 0.28.
Keypoints rilevati	7.1	13.4	L'M3 e l'M1 aumentano
Jitter (Instabilità)	0.045	0.005	Il jitter è sull'M3 è eccezionale.

MoveNet MultiPose

Il terzo modello in ordine di sviluppo all'interno del progetto, estende la capacità di MoveNet al rilevamento **multi-soggetto**, capace di tracciare fino a **6 persone** contemporaneamente. La base è il modello **Lightning** per contenere il costo computazionale durante la **detection multipla**, e viene configurata per l'esecuzione in locale tramite i file estratti dall'archivio.

A differenza delle varianti Single Pose con dimensione fissa, MultiPose richiede un resize **proporzionale con larghezza fissa a 256px** e **altezza calcolata dinamicamente**. Il codice legge prima le dimensioni native del video con: `cap.get(cv2.CAP_PROP_FRAME_WIDTH/HEIGHT)`, calcola il **fattore di proporzione e arrotonda** l'altezza target al **multiplo di 32 più vicino**, requisito architetturale del modello per evitare artefatti di bordo nei **livelli convoluzionali**. Il **tensore** viene castato a **int32** con `tf.cast(tf.expand_dims(...), dtype=tf.int32)`.

L'**output grezzo** ha forma [1, 6, 56], dove i primi **51 valori per persona** **corrispondono ai 17 keypoints x 3 coordinate [y, x, conf]**, mentre i **5 valori rimanenti** rappresentano le **bounding box della persona**. Il codice seleziona i **solli keypoints** con `results['output_0'].numpy()[::,51]` e li ridimensiona a (6, 17, 3) con `.reshape((6,17,3))`, ottenendo un array iterabile per persona.

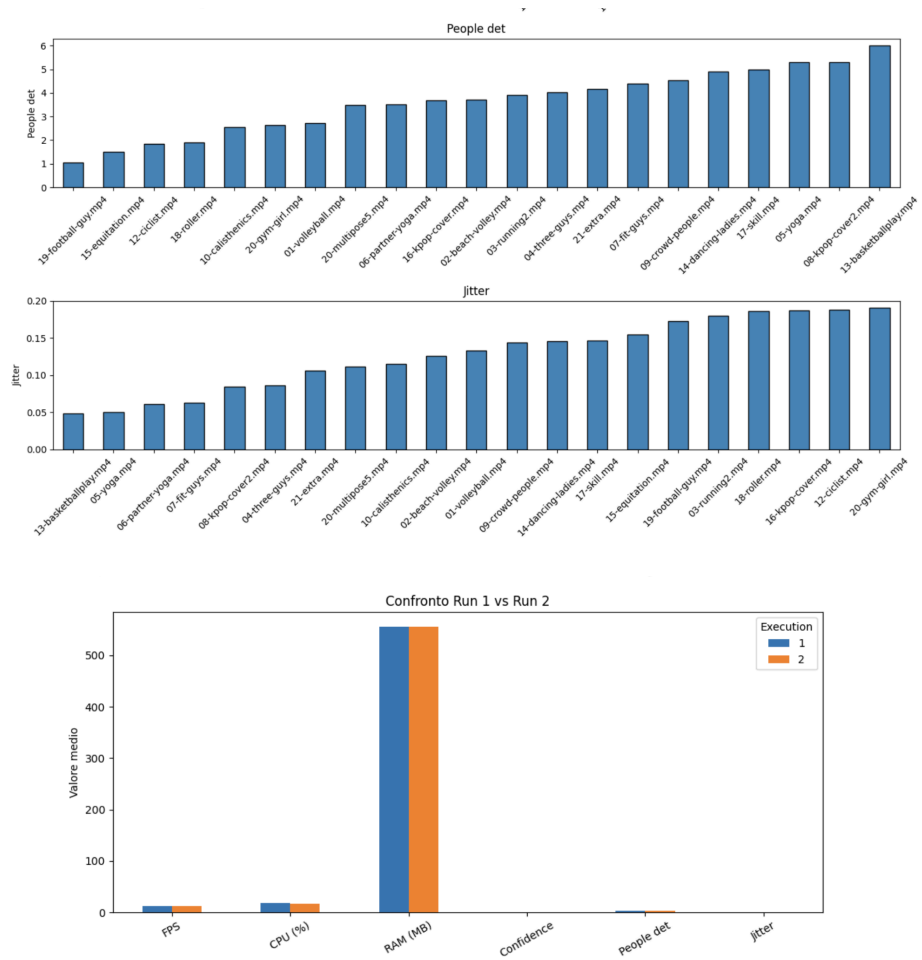
La funzione `loop_through_people()` itera su tutte e 6 le slot di output, chiamando `draw_connections()` e `draw_keypoints()` per ciascuna persona. Analogamente `render_skeleton()` costruisce lo scheletro nero iterando su tutte le relative persone. Le persone non presenti nel frame, producono **slot con tutti i valori di confidenza prossimi a 0**, che vengono filtrati automaticamente dalle soglie.

Il **post-processing** estende quello SinglePose considerando tutte le 6 slot. Conta le **persone rilevate** come quelle che hanno **almeno un keypoint sopra soglia**, calcola la **confidenza media escludendo le slot vuote** e come **jitter** la distanza media tra i **keypoint di tutte le persone tra frame consecutivi**.

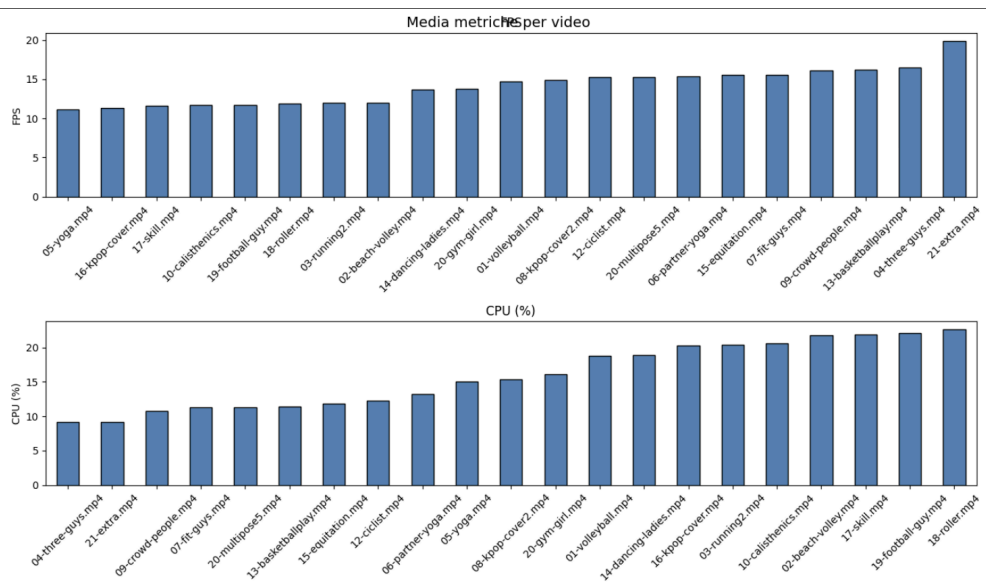
Comparazione prestazioni tra M1 e M3

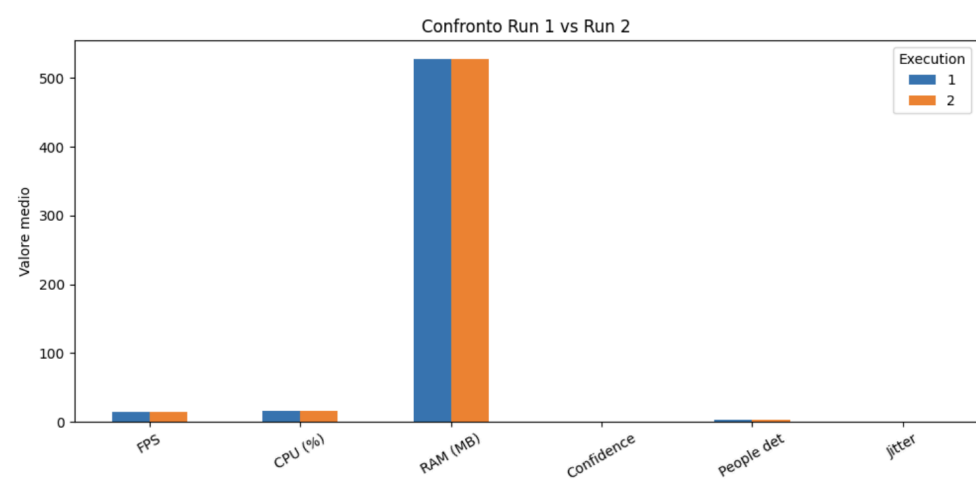
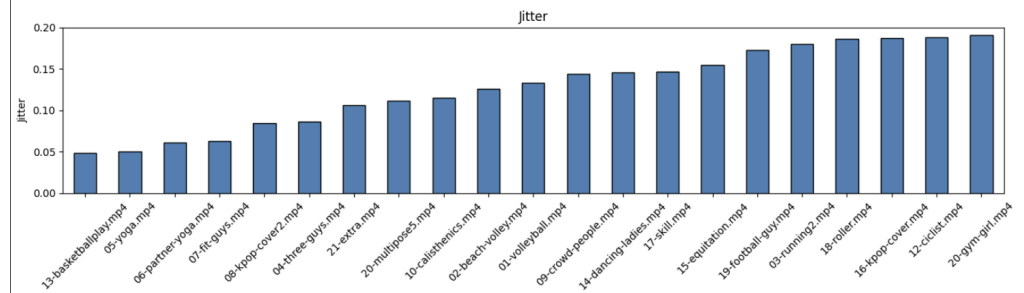
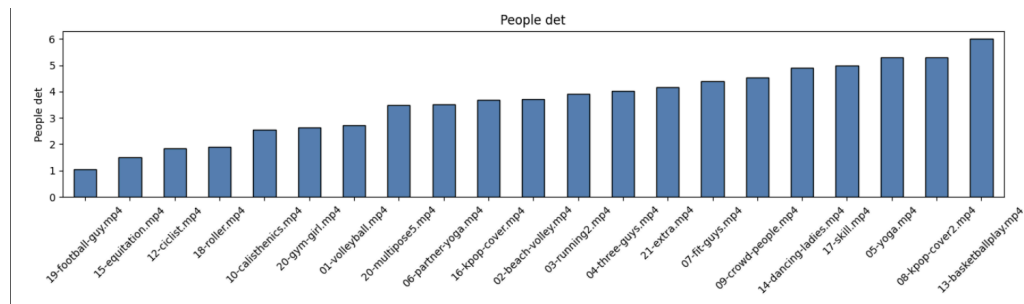
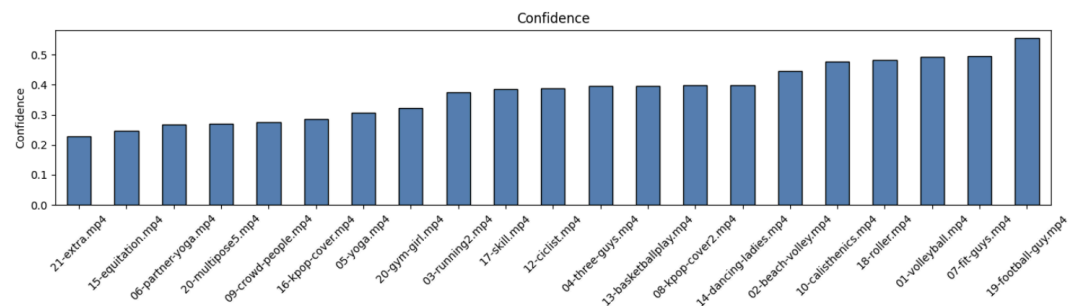
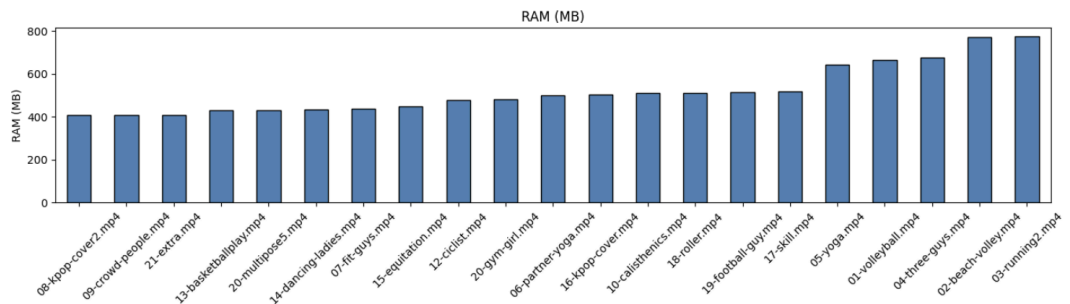
Grafici M1





Grafici M3





Metrica	MacBook Air M1	MacBook Air M3	Differenze
FPS (Frame per Second)	~12.47	~14.12	L'M1 guadagna circa 1.5 FPS.
RAM (Utilizzo medio)	~555.40MB	~528.12 MB	L'M3 consuma il 27 MB di RAM in meno.
CPU (Utilizzo medio)	~17.66%	~16.10%	L'M3 consuma circa il 9% in meno di CPU
Confidence (Fiducia)	0.39	0.39	L'M3 mantiene la confidenza molto alta e M1 scende a 0.28.
Persone rilevate	3.62	3.62	
Jitter (Instabilità)	0.124	0.124	Il jitter è significativamente più alto rispetto ai modelli single-pose analizzati precedentemente (0,124 e 0.025 per M3). Questo accade perché questo modello deve dividere le proprie risorse per tracciare più soggetti.

MediaPipe Holistic

Il quarto modello in ordine di sviluppo all'interno del progetto, implementa la **pipeline Holistic di MediaPipe**, che integra in un **singolo grafo di calcolo quattro modelli distinti: POSE, VOLTO, MANI** per un totale di **543 landmark** per soggetto.

MediaPipe Holistic **non richiede un resize o un cast del tensore**, il modello infatti vive all'interno di un **context manager** (ovvero a un costrutto specifico di Python gestito tramite un'istruzione **with** che serve a gestire il ciclo di vita delle risorse): **with mp_holistic.Holistic(...) as holistic**, che inizializza e dealloca correttamente le risorse.

Le funzioni della pipeline ricevono **l'istanza holistic** come parametro invece dell'interprete **TFLite**.

Fase di PRE-processing

La funzione **pre_processing(frame)** si riduce a una sola operazione, ovvero la conversione **BGR a RGB** con **cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)**.

MediaPipe richiede RGB in ingresso e gestisce internamente qualsiasi ridimensionamento necessario.

Inferenza

La funzione **make_detections(holistic, image)** chiama **holistic.process(image)** che esegue in sequenza tutti i sotto-modelli della pipeline Holistic.

L'oggetto **results** restituito contiene quattro attributi: **pose_landmarks (33 punti)**, **face_landmarks (468 punti)**, **left_hand_landmarks** e **right_hand_landmarks (21 punti ciascuna)**.

Quando un componente non è rilevato, il relativo attributo è **None**. Dopo il processo, il frame viene riconvertito in BGR per il rendering con OpenCV.

Fase di POST-processing

A differenza di MoveNet, che sfrutta un campo di **confidence**, MediaPipe espone la qualità del tracking tramite la **visibility** di ogni landmark (valore **tra 0 e 1**). Questo parametro misura **QUANTO UN PUNTO SIA VISIBILE** e non occluso; nel nostro progetto, la media delle **visibility dei 33 landmark** di posa funge da indicatore di affidabilità globale.

Oltre alla qualità del singolo frame, il sistema valuta la coerenza del movimento calcolando il **JITTER**, ovvero la **fluttuazione** delle coordinate [x, y] tra i fotogrammi successivi.

Rendering

La funzione **rendering()** disegna separatamente i quattro componenti sia sul frame originale che sullo schermo nero, usando **mp_drawing.draw_landmarks()** con specifiche di stile dedicate per ciascun componente.

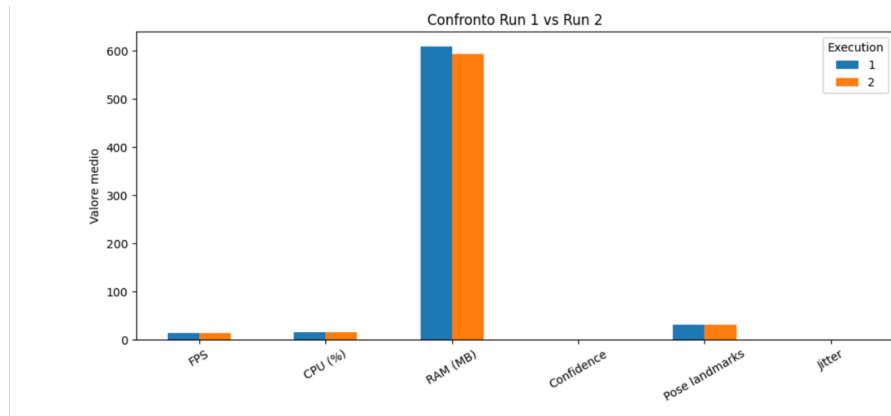
I contorni del volto sono disegnati **in verde (FACEMESH_CONTOURS)**, le mani in tonalità calde (**HAND_CONNECTIONS**) con **colori distinti per destra e sinistra**, e la posa in **arancione/viola (POSE_CONNECTIONS)**.

L'uso di **DrawingSpec** permette di controllare colore, spessore e raggio di ogni componente visivo.

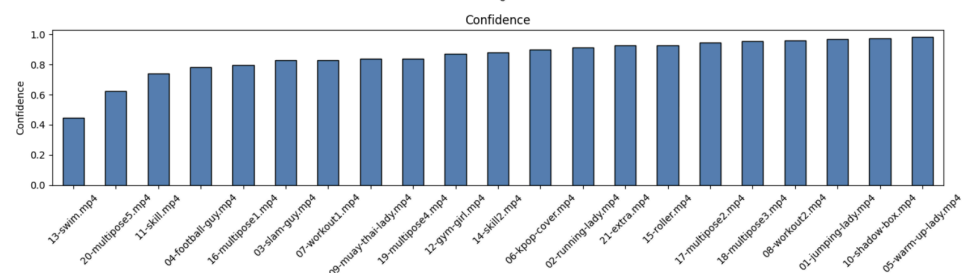
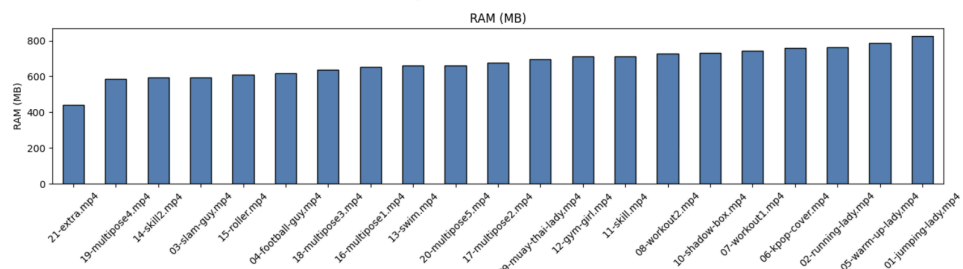
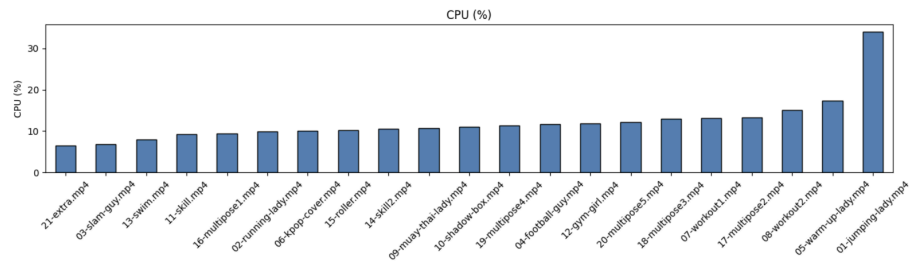
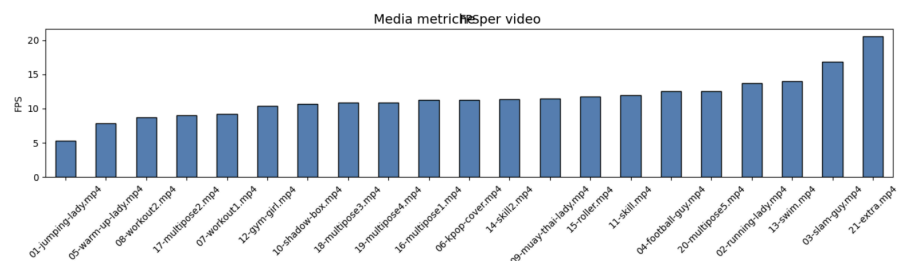
Comparazione prestazioni tra M1 e M3

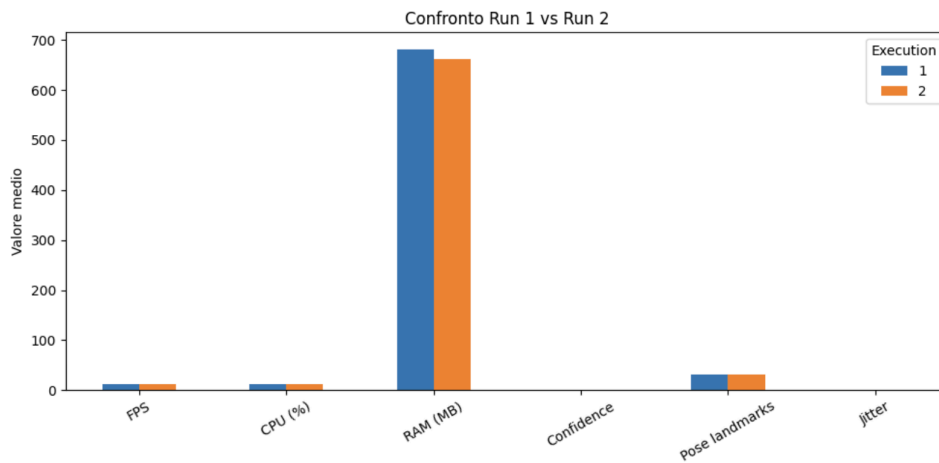
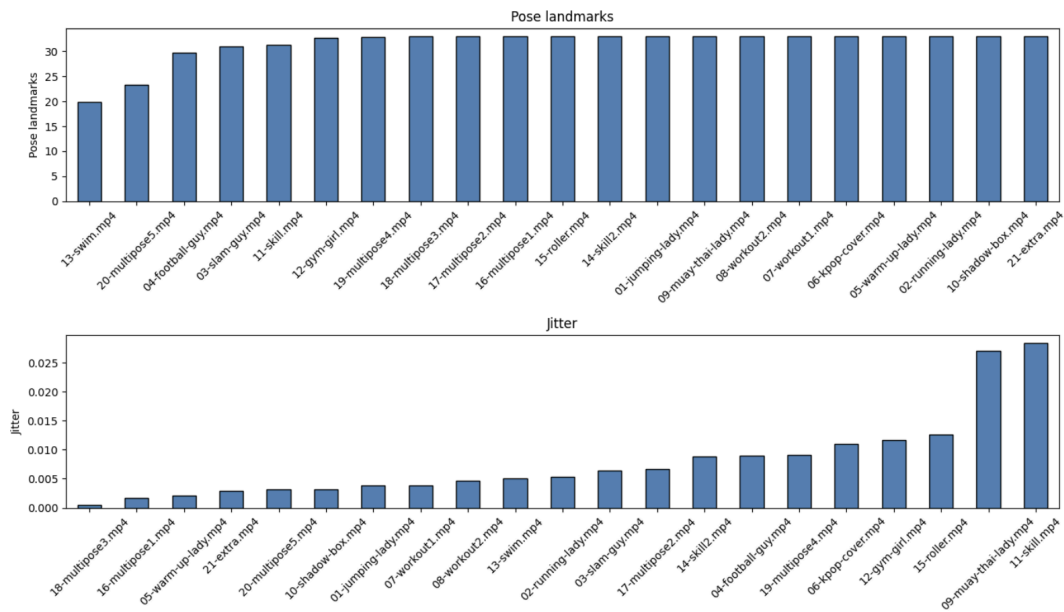
Grafici M1





Grafici M3





Metrica	MacBook Air M1	MacBook Air M3	Differenze
FPS (Frame per Second, media)	~11.85	~15.42	Nonostante Holistic sia più difficile da processare, l'M3 riesce a mantenere una media di oltre 15 FPS, toccando a volte 21 FPS. L'M1 invece rimane attorno agli 11-12 FPS. Per Holistic superare la soglia di 15 FPS è il limite minimo per

			percepire un movimento naturale.
RAM (Utilizzo medio)	~595.60 MB	~468.20 MB	L'M3 gestisce il modello con 130 MB in meno rispetto ad M1.
CPU (Utilizzo medio)	~28.4%	~9.2%	Mentre M1 deve impiegare quasi il 30% delle sue risorse, l'M3 si stabilizza intorno al 9%, poiché delega quasi tutto il lavoro ai core specializzati del Neural Engine.
Pose Landmarks	33/33	33/33	Entrambi rilevano 33 landmark
Confidence (Fiducia)	~0.92	~0.92	Hanno entrambi una confidenza altissima
Jitter (Instabilità)	0.0035	0.0028	Il jitter è più basso nel M3 poiché la maggiore potenza di calcolo permette di eseguire più iterazioni di rifinitura nello stesso lasso di tempo rendendo il tracking delle mani e dita meno preciso.

MediaPipe + YOLOv8

L'ultimo modello in ordine di sviluppo all'interno del progetto, una pipeline **ibrida a due livelli**, progettata combinando algoritmi di **Object Tracking e Pose Estimation per monitorare soggetti umani in tempo reale**.

Viene impiegato quindi **YOLOv8** che rileva le persone nel **frame intero e calcola le bounding box**, mentre **MediaPipe** stima la posa su ciascun **crop** individuale.

Configurazione dei due modelli

YOLOv8 viene caricato con **YOLO('yolov8n.pt')**, scaricando automaticamente i pesi del modello Nano alla prima esecuzione.

MediaPipe viene configurato tramite la funzione **build_mp_options()** che costruisce un **oggetto PoseLandmarkerOptions** con modalità **IMAGE (frame singolo)**, soglie di **confidenza a 0.3** e **num_poses = 2**.

Il device per YOLO viene selezionato automaticamente, se disponibile viene usato il backend **MPS** (Metal Performance Shaders, la GPU Apple Silicon) tramite **torch.backends.mps.is_available()**, altrimenti si ricade sulla CPU.

Fase di PRE-processing

La funzione **pre_processing(frame)** crea **due canvas**, uno **colored**, il quale è una copia del frame originale su cui disegnare scheletri e bounding box e il **black**, costituito da un **array di zeri delle stesse dimensioni** per il rendering isolato. Questo evita di sovrascrivere il frame originale necessario a YOLO.

YOLO detection

La funzione **yolo_detection()** chiama **yolo_model.track()** con **persist = True** per mantenere l'**ID** di tracking **persistenti e coerenti** tra frame consecutivi.

Il parametro **classes = [0]** limita il rilevamento alla sola **classe 0** (persone nel dataset COCO).

Il risultato restituisce le **coordinate dei bounding box** in formato **xyxy** (ovvero **x_min, y_min, x_max, y_max**) e gli ID associati.

Per ottimizzare l'accuratezza della stima della posa, il sistema non si limita a ritagliare l'area indicata dalla bounding box di YOLOv8. Viene invece applicata una logica di **padding e clipping**.

Ciascun **box viene espanso** del **10%** su tutti i lati. Questa operazione è fondamentale per evitare il **troncamento involontario degli arti** che potrebbero trovarsi in prossimità dei bordi della rilevazione originale. Inoltre, le **coordinate espansive** vengono sottoposte a una funzione di **clipping**, che garantisce che le coordinate della bounding box rimangano **sempre all'interno del perimetro del frame video** assicurando che il box non ecceda i limiti fisici del frame video.

MediaPipe per crop

La funzione **mediapipe_pose_estimation()** itera su ogni persona rilevata da YOLO. Per ogni soggetto, viene estratto un segmento del frame originale tramite la tecnica dello **SLICING: NumPy: frame[y1c:y2c, x1c:x2c]**. Questa operazione permette di **isolare** il soggetto riducendo drasticamente il rumore di fondo e il carico computazionale per l'algoritmo successivo.

Poiché **OpenCV** acquisisce i frame in formato **BGR**, il **crop** viene convertito in **RGB** e lo incapsula nell'oggetto **mp.Image**.

Questo passaggio si intreccia con l'architettura di **MediaPipe Tasks** che ottimizza l'esecuzione dei modelli tramite metodo **mp_lm.detect()**.

I landmark restituiti hanno **coordinate normalizzate [0,1]** rispetto al crop.

Per rendere questi dati coerenti con il frame originale, la funzione **draw_skeleton_on_canvas()** applica una **TRASFORMAZIONE BASATA SU OFFSET E SCALA**.

La **proiezione delle coordinate locali** $lm.x$, $lm.y$ in coordinate globali abs_x , abs_y avviene secondo la logica:

$$abs_x = \frac{(lm.x \cdot scale_x) + x_{off}}{w}$$

Dove $lm.x \cdot scale_x$ **converte la coordinata normalizzata del landmark in un valore pixel assoluto** e il $+x_{off}$ **applica l'offset orizzontale**, ovvero la posizione X di partenza della bounding box traslando il punto nella corretta posizione.

Il $/w$ esegue una **nuova normalizzazione** dividendo per la larghezza totale del frame.

Fase di POST-processing

La pipeline ibrida introduce un post-processing più articolato con **due misure distinte di instabilità**: il **jitter_people** misura la **variazione del numero di persone** rilevate **tra frame consecutivi**, utilizzato come indicatore di instabilità del detector YOLO.

Mentre il **jitter_kp** misura lo **spostamento medio delle coordinate dei keypoint** di MediaPipe tra frame, calcolato abbinando le persone per posizione.

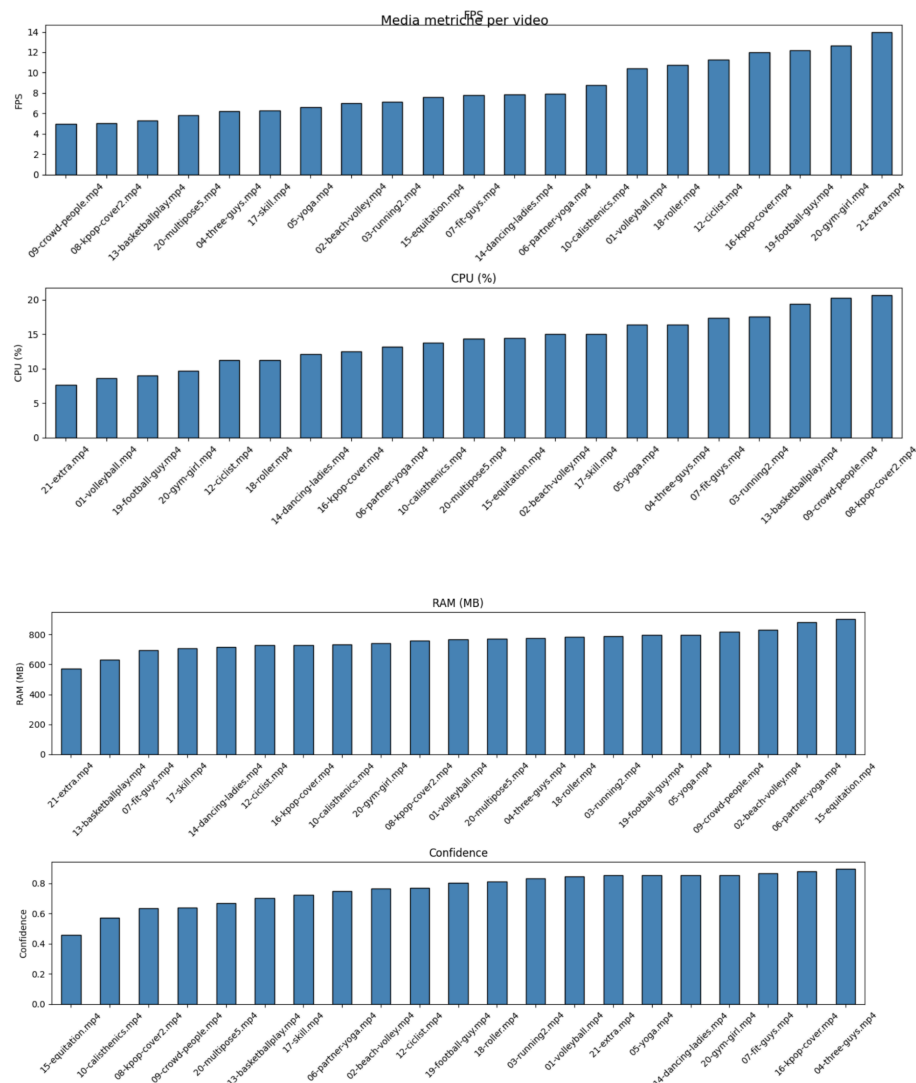
Rendering

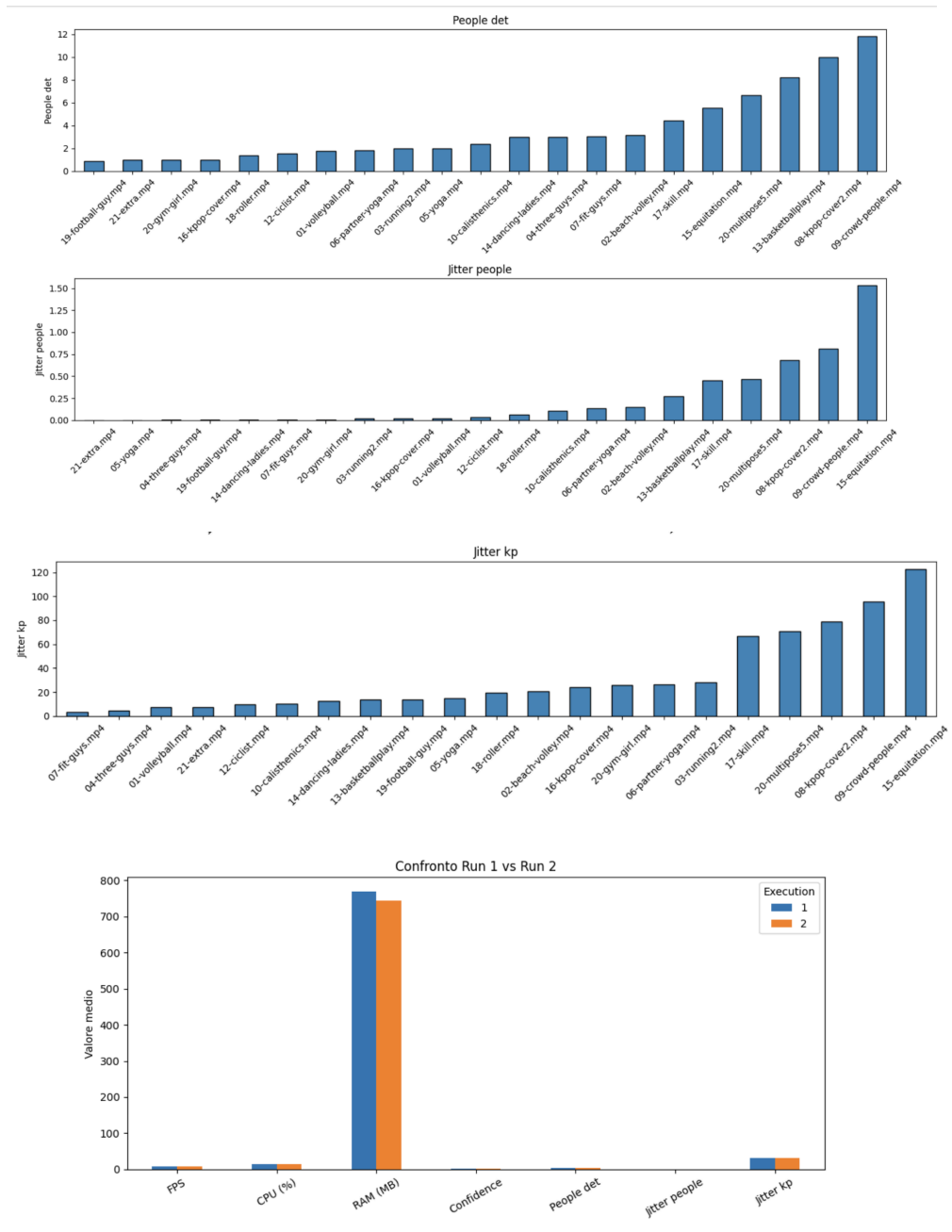
La funzione **rendering()** utilizza la funzione **get_display_size()** per calcolare la **dimensione di visualizzazione** con **larghezza MASSIMA fissa a 720 px**, ridimensionando proporzionalmente entrambi i canvas con **cv2.resize()**.

Il font del testo in sovrapposizione viene **scalato dinamicamente** con **font_scale = dw / 640**, usando **640px come larghezza di riferimento** (risoluzione standard VGA), così i testi rimangono leggibili indipendentemente dalla risoluzione del video.

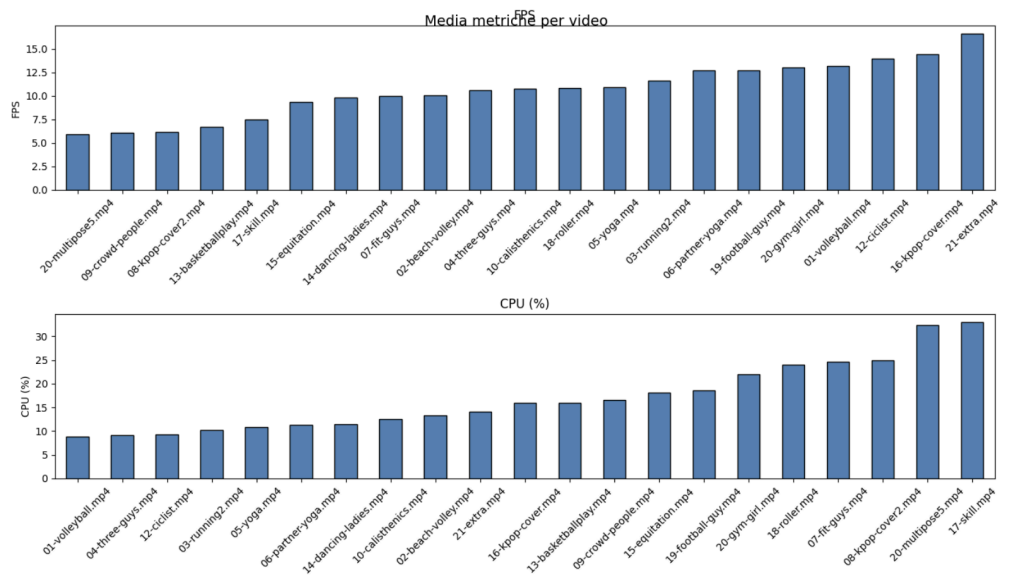
Comparazione prestazioni tra M1 e M3

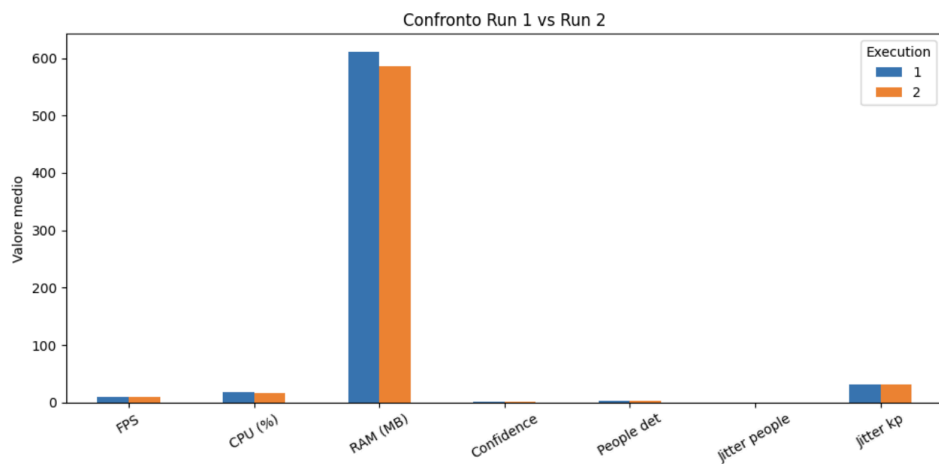
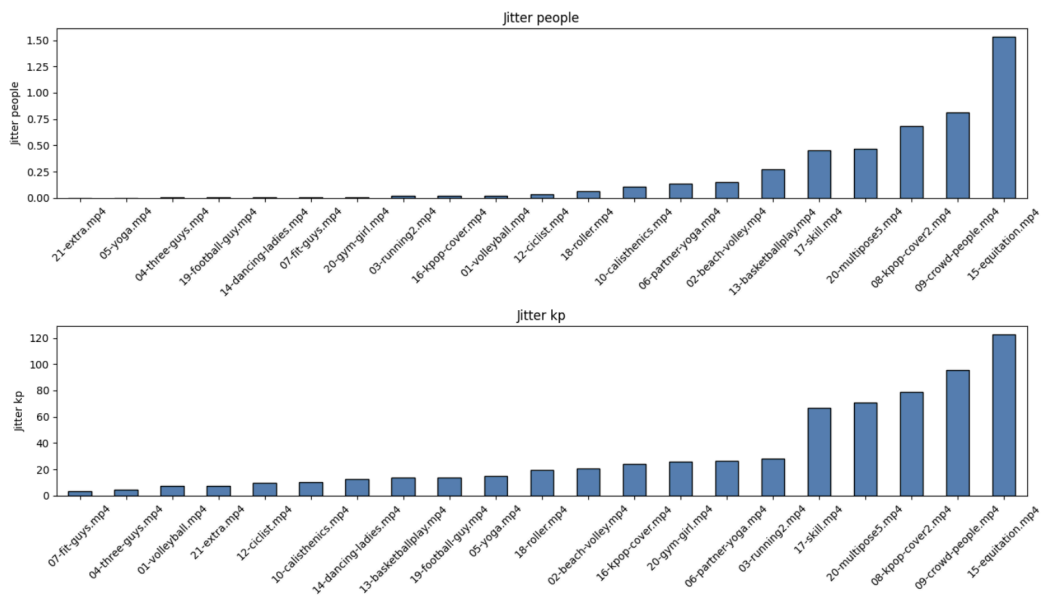
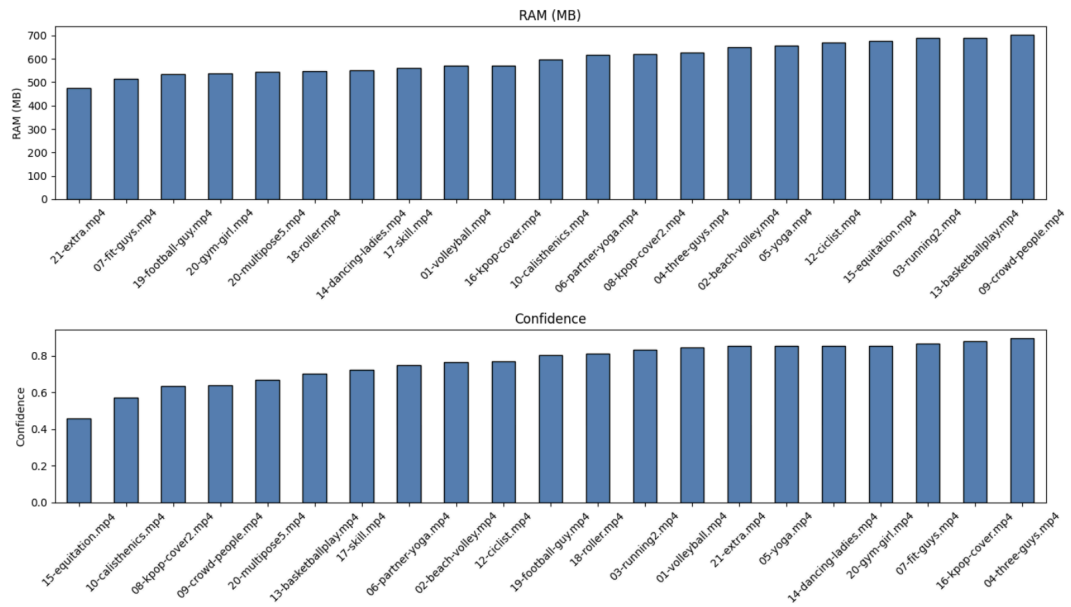
Grafici M1





Grafici M3





Metrica	MacBook Air M1	MacBook Air M3	Differenze
FPS (Frame per Second, media)	~6.50 -7.00	~10.57	L'M3 rende YOLO quasi utilizzabile per un'anteprima fluida, l'M1 rimane sotto i 7 FPS producendo immagini "a scatti"
RAM (Utilizzo medio)	~780 MB	~598.81 MB	
CPU (Utilizzo medio)	~35.2%	~16.91%	M1 mostra un importante fatica, M3 dimostra di avere un'architettura più portata per i modelli per l'Object Detection
Confidence (Fiducia)	0.75	0.76	
Jitter (Instabilità)	~45.0	31.17	Il jitter dei keypoints ha dei keypoints meno stabili rispetto ad Holistic, nell'M3 è più contenuto permettendo una rilevazione più precisa.

Analisi conclusiva per il confronto M1 e M3

In conclusione, l'M3 consuma mediamente la metà della CPU rispetto al chip M1, garantendo sempre un incremento di FPS tra il 20% e il 50%. I jitter sono sistematicamente inferiori sul M3, producendo dati potenzialmente più puliti.

Bibliografia

Bibliografia

Fonti : Introduzione

“Analisi chip M1: *Apple presenta il chip M1*. Apple Newsroom” :

<https://www.apple.com/it/newsroom/2020/11/apple-unleashes-m1/>

“Analisi chip M3: Apple svela M1, M3 Pro e M3 Max: i chip più avanzati per un personal computer.” :

<https://www.apple.com/it/newsroom/2023/10/apple-unveils-m3-m3-pro-and-m3-max/>

“Descrizione Apple Silicon” :

https://it.wikipedia.org/wiki/Apple_Silicon

“Architettura dei sistemi di elaborazione: analisi delle architetture ARM e RISC, struttura fisica del calcolatore, ciclo Fetch-Decode-Execute, tecniche di pipelining, parallelismo e architetture scalari”: **Dispense del Corso**.

Fonti : TensorFlow - MoveNet

“Introduzione a Tensor Flow” :

https://www.tensorflow.org/guide/basics?hl=it#modules_layers_and_models

<https://www.nvidia.com/en-us/glossary/tensorflow/>

<https://medium.com/@RobuRishabh/introduction-to-tensorflow-bbb13c3675e7>

"Gradienti e Automatic Differentiation" :

<https://www.tensorflow.org/guide/autodiff>

<https://tf.wiki/en/basic/basic.html>

<https://www.geeksforgeeks.org/deep-learning/tf-gradienttape-in-tensorflow/>

“Introduzione alle Variabili” :

<https://tensorflow.rstudio.com/guides/tensorflow/variable>

“Introduzione a Movenet” :

<https://blog.tensorflow.org/2021/05/next-generation-pose-detection-with-move-net-and-tensorflowjs.html>

<https://www.tensorflow.org/hub/tutorials/movenet?hl=it>

“Human Pose Tracking with Movenet” :

<https://ijetms.in/Vol-9-issue-2/Vol-9-Issue-2-58.pdf>

“MoveNet Lightning e Thunder” :

<http://storage.googleapis.com/movenet/MoveNet.SinglePose%20Model%20Card.pdf>

“DataSet” :

<https://medium.com/axinc-ai/movenet-pose-estimation-for-video-with-intense-motion-2b92f53f3c8>

“Cropping” :

<https://www.tensorflow.org/hub/tutorials/movenet>

“MoveNet Multipose” :

<https://storage.googleapis.com/movenet/MoveNet.MultiPose%20Model%20Card.pdf>

Fonti : Librerie

“NumPy documentation” :

<https://numpy.org/doc/2.4/>

“opencv-python” :

<https://pypi.org/project/opencv-python/>

“OpenCV - Documentation” :

<https://docs.opencv.org/4.13.0/>

“matplotlib 3.5.3 documentation” :

<https://matplotlib.org/3.5.3/index.html>

“psutil documentation” :

<https://psutil.readthedocs.io/stable/>

“os — Miscellaneous operating system interfaces” :

<https://docs.python.org/3/library/os.html>

“glob - Unix style pathname pattern expansion” :

<https://docs.python.org/3/library/glob.html>

“pandas” :

<https://pandas.pydata.org>

“pandas: A Powerful Python Data Analysis Toolkit” :

<https://pypi.org/project/pandas/>

“time - Time access and conversions” :

<https://docs.python.org/3/library/time.html>

“pytorch : tensors and dynamic neural networks in Python with GPU acceleration” :

<https://github.com/pytorch/pytorch>

Fonti : YOLO - YOLOv8

“Ultralytics YOLO Docs” :

<https://docs.ultralytics.com/#where-to-start>

“Ultralytics YOLOv8” :

<https://docs.ultralytics.com/models/yolov8/>

“What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector” :

<https://arxiv.org/html/2408.15857v1>

Fonti : Video Tutorial

“Nicholas Renotte, Pose Estimation with the Fastest Python Deep Learning Model | MoveNet Lightning” :

<https://www.youtube.com/watch?v=SSW9LzOJSus>

“Nicholas Renotte, Tensorflow Multi-Person Pose Estimation with Python | MoveNet Multipose” :

<https://www.youtube.com/watch?v=KC7nJtBHBqg&t=387s>

“Nicholas Renotte, AI Face Body and Hand Pose Detection with Python and Mediapipe” :

<https://www.youtube.com/watch?v=pG4sUNDOZFg>